4.4 Graphics Processing Units (GPU)

Graphical Processing Units

- **Basic idea:**
  - **Heterogeneous** execution model
    - CPU is the *host*, GPU is the *device*
  - Develop a **C-like programming language** for GPU, namely, **CUDA**
  - Unify all forms of GPU parallelism as **CUDA thread**
  - Programming model is "**Single Instruction Multiple Thread (SIMT)** "

NVIDIA GPU Architecture

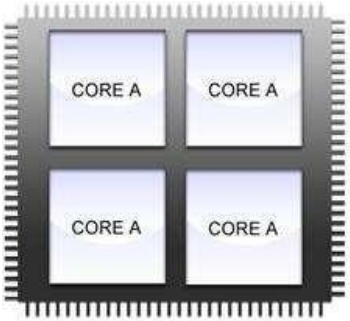
- **Similarities to vector machines:**
  - Works well with data-level parallel problems
  - Scatter-gather transfers
  - Mask registers
  - Large register files
- **Differences:**
  - Uses multithreading to hide memory latency
  - Has many functional units, as opposed to a few deeply pipelined units like a vector processor

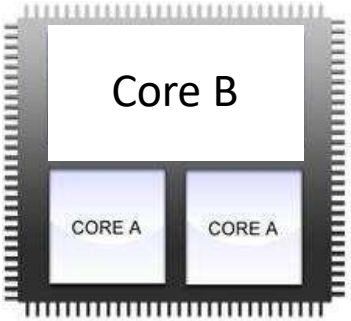
Heterogeneous computing

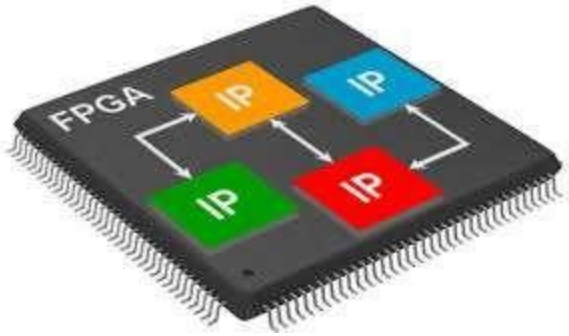
Heterogeneous computing

- **Homogeneous computing**
  - uses one or more processor of the **same architecture** to execute an application.

- **Heterogeneous computing**
  - uses **a suite of processor architectures** to execute an application
  - applying tasks to architectures to which they are well-suited
  - yielding performance improvement as a result



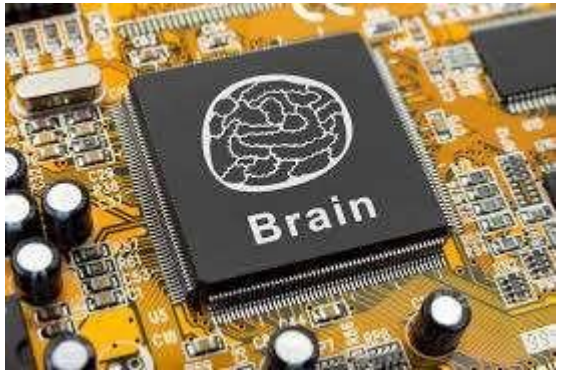




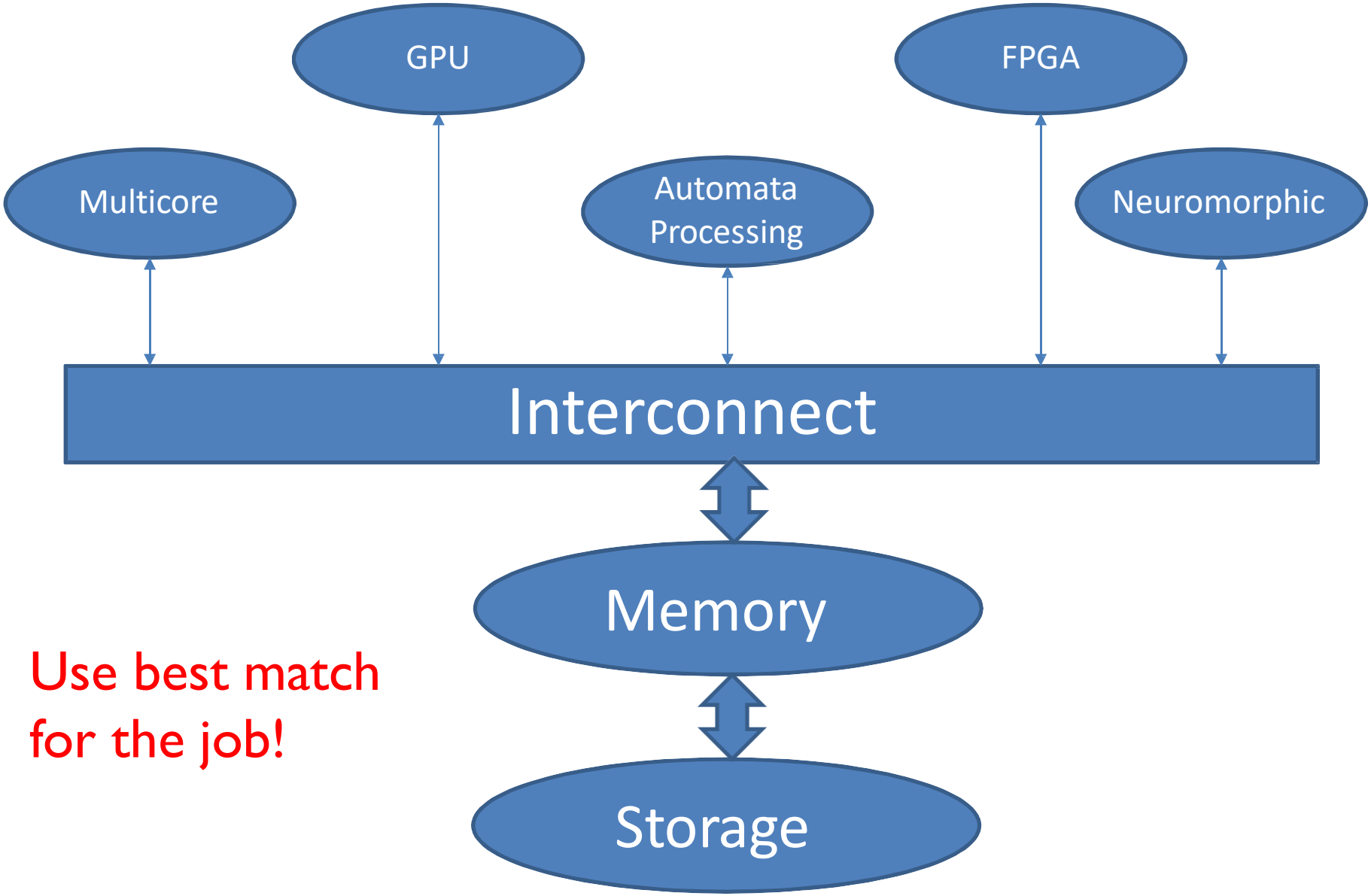
Heterogeneity Everywhere







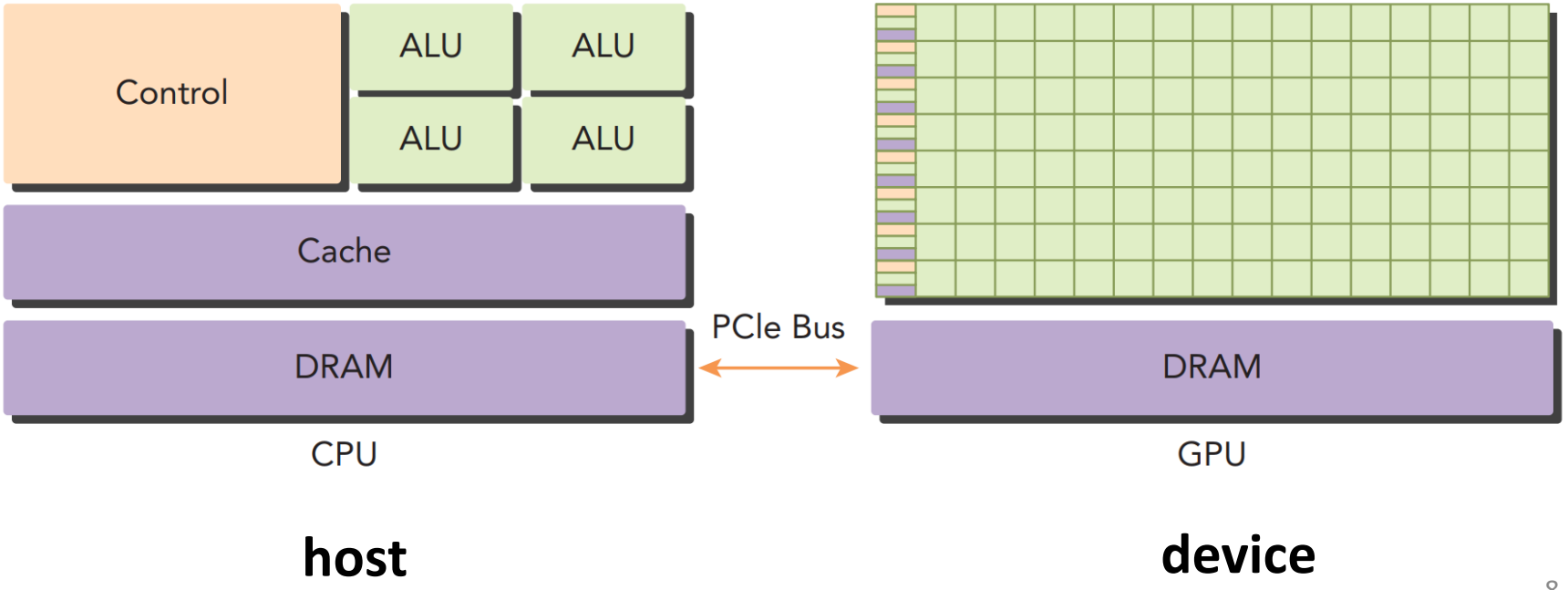
Neuromorphic Chips



Use best match for the job!

Heterogeneous Architecture

- A typical heterogeneous compute node nowadays consists of two multicore CPU sockets and two or more many-core GPUs



Heterogeneous Architecture

- A heterogeneous application consists of two parts:
  - **Host code**
  - **Device code**

- Host code runs on CPUs and device code runs on GPUs.

- GPUs are used to accelerate the execution of this portion of data parallelism.
  - referred to as a ***hardware accelerator***

NVIDIA's GPU computing platforms

- Tegra
  - For mobile and embedded devices
- GeForce
  - for consumer graphics
- Quadro
  - for professional visualization
- Tesla：
  - for datacenter parallel computing
  - Tesla (2006)→Fermi (2010)→Kepler (2012)→Maxwell (2014) →Pascal (2016)→Volta (2017)→Turing (2018) →Ampere (2020) →Hopper (2022) →Ada Lovelace(2023) → Blackwell (2024) → Rubin(2026) & Feynman(2028)

GPU capability

- Number of CUDA cores
- Memory size
- Peak computational performance
- Memory bandwidth

	"Kepler" K20	"Kepler" K40	"Maxwell" M40	Pascal P100	Volta V100
CUDA cores	2496	2880	3072	3584	5120
Cores per SM	192	192	128	64	64
Single Precision	3.52 Tflops	4.29 Tflops	7.0 Tflops	9.5 Tflops	15 Tflops
Double Precision	1.17 TFlops	1.43 Tflops	0.21 Tflops	4.7 Tflops	7.5Tflops
Memory BW	208 GB/s	288 GB/s	288 GB/s	720 GB/s	900 GB/s
Memory size	5 GB	12 GB	12 GB	12/16 GB	16 GB

Compute Capability

- NVIDIA uses a special term, compute capability, to describe hardware versions of GPU accelerators
  - denoted by X.Y
  - devices with the same **major revision number** are of the same core architecture
- Telsa: 1 → Fermi: 2 → Kepler: 3 → Maxwell: 5 → Pascal: 6 → Volta/Turing: 7 → Ampere: 8 → Hopper: 9 → Blackwell: 12 …

GPU	COMPUTE CAPABILITY
Tesla K40	3.5
Tesla K20	3.5
Tesla K10	3.0
Tesla C2070	2.0
Tesla C1060	1.3

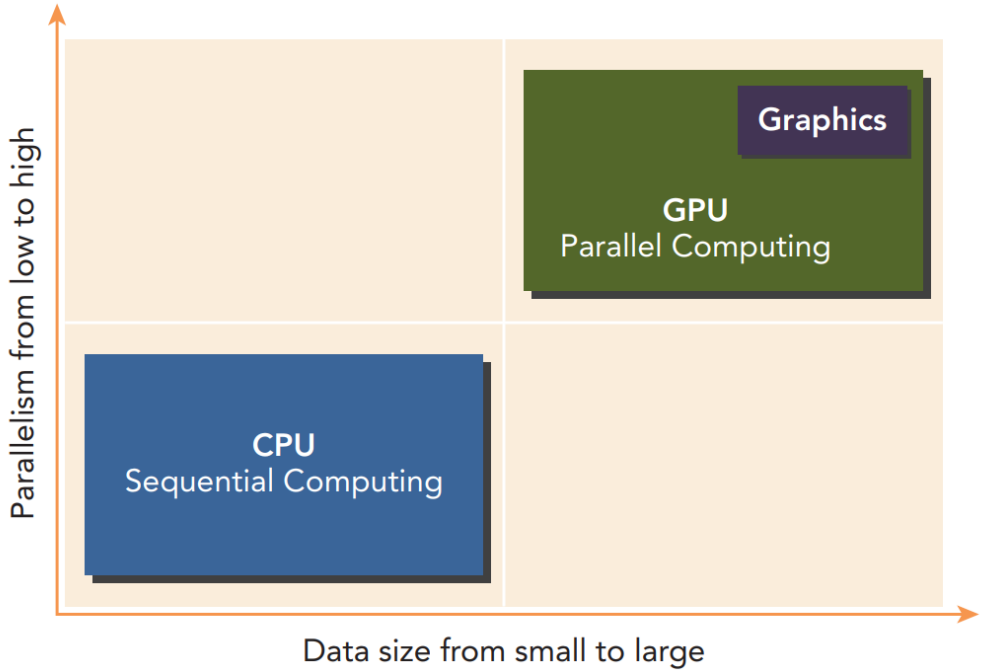
Compute Capability

Technical specifications	Compute capability (version)																		
	1.0	1.1	1.2	1.3	2.x	3.0	3.2	3.5	3.7	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6
Maximum number of resident grids per device (concurrent kernel execution)	t.b.d.				16		4	32				16	128	32	16	128	16	128		
Maximum dimensionality of grid of thread blocks	2				3															
Maximum x-dimension of a grid of thread blocks	65535				$2^{31} - 1$															
Maximum y-, or z-dimension of a grid of thread blocks	65535																			
Maximum dimensionality of thread block	3																			
Maximum x- or y-dimension of a block	512				1024															
Maximum z-dimension of a block	64																			
Maximum number of threads per block	512				1024															
Warp size	32																			
Maximum number of resident blocks per multiprocessor	8					16				32								16	32	16
Maximum number of resident warps per multiprocessor	24		32		48			64										32	64	48

https://en.wikipedia.org/wiki/CUDA

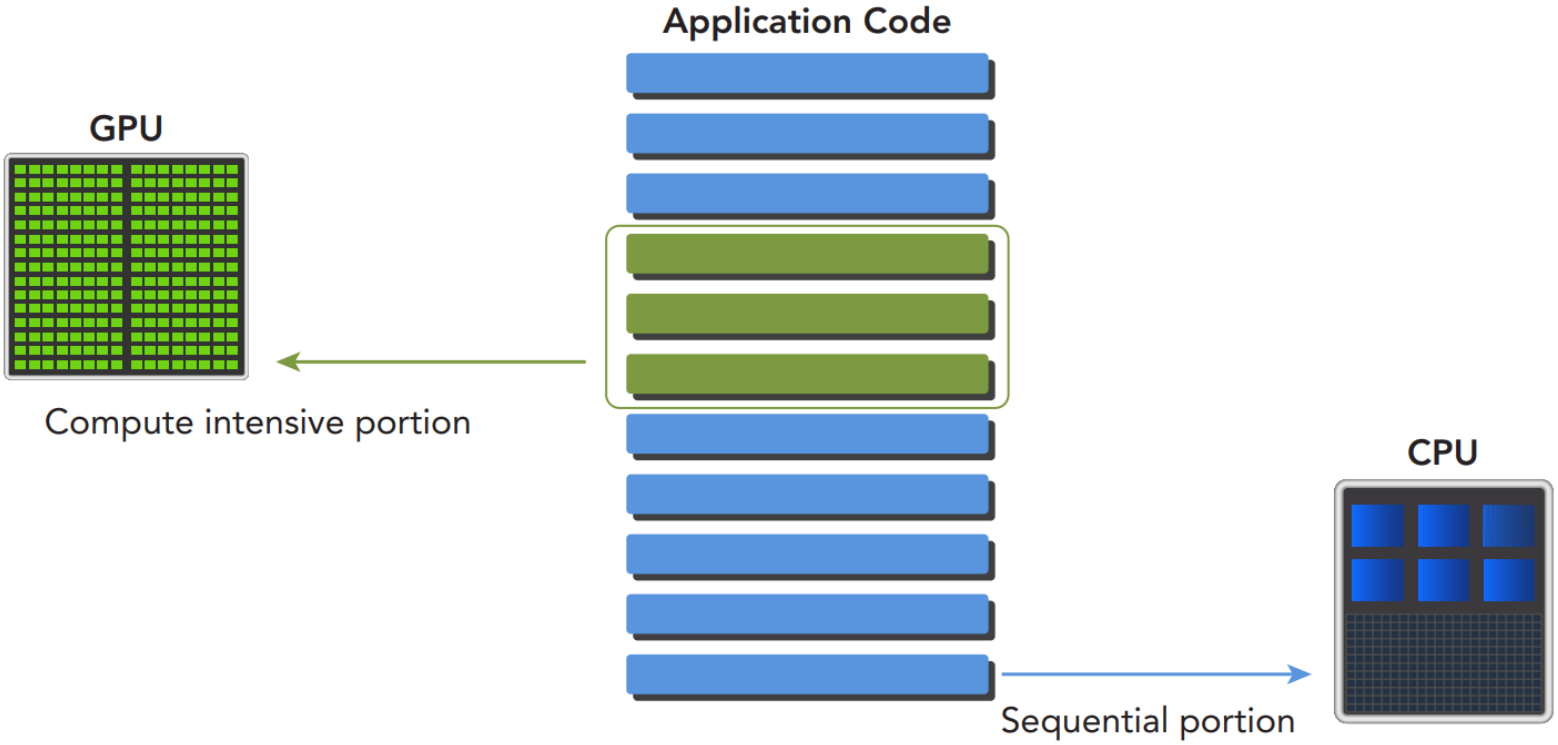
Paradigm of Heterogeneous Computing

- GPU computing is not meant to replace CPU computing.
  - CPU computing is good for control-intensive tasks
  - GPU computing is good for data-parallel computation-intensive tasks.



Paradigm of Heterogeneous Computing

- Need to use both CPU and GPU for your application
  - executing the sequential parts or task parallel parts on the CPU and intensive data parallel parts on the GPU



CPU Thread vs. GPU Thread

- **CPU Thread**
  - Threads on a CPU are generally heavyweight entities.
  - The OS must swap threads on and off CPU execution channels to provide multithreading capability.
  - A CPU with 4 quad core processors can run only 16 threads concurrently, or 32 if the CPUs support hyper-threading.

- **GPU Thread**
  - Threads on GPUs are extremely lightweight.
  - In a typical system, thousands of threads are queued up for work.
  - Modern NVIDIA GPUs can support up to 1,536 active threads concurrently per multiprocessor.

History of GPU

History of GPUs …
How did they evolve?



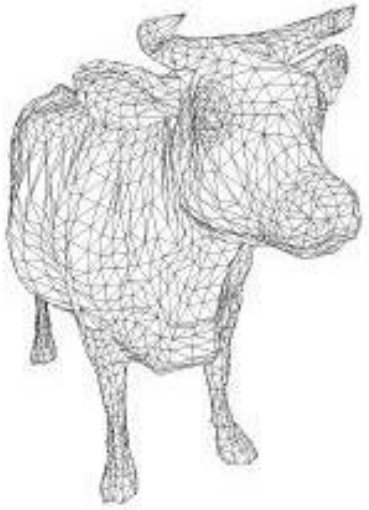
Why Looking at GPU History?

- Looking at how things evolved can highlight future directions.

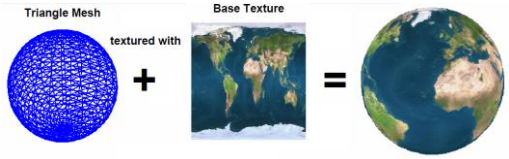
- Some of the current architecture decisions won't make sense without historical perspective.

Graphic Elements

- ***Vertex:*** A point with 3D coordinate and color.
- ***Polygons:*** composed of vertices and edges.
- ***Mesh:*** A grid of polygons to represent an object.
- ***Objects:*** Any 2D or 3D entity whose shape could be represented by mesh of arbitrary polygons.
- ***Primitive:*** Classical geometric shapes can be directly used as primitives (e.g. point, line, cube, cylinder, sphere...) to make parts of objects.



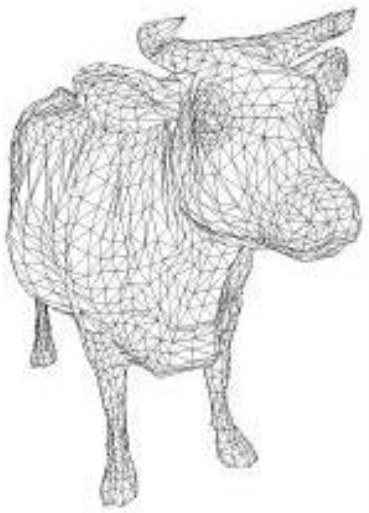
F. Durand, "A Short Introduction to Computer Graphics", MIT Laboratory for Computer Science



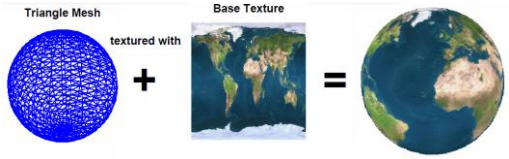
R. Fernando, C. Zeller, "Programming Graphics Hardware Programming Hardware", NVIDIA.

Graphic Elements

- ***Texture:*** An image that is mapped on the surface of polygons on objects to provide a concept of specific material.
- ***Color:*** The color of each pixel is described by three numbers for intensity of main colors: red, green, and blue, e.g. (255, 0, 255).
- ***Pixel:*** A computer image is represented by an array of points called pixel with its own color and coordinate (address).
- ***Resolution:*** The number of pixels in an image determines the resolution of the image, e.g. 320x200, 2560x2048.
- ***Fragment:*** All necessary data needed to generate a single pixel of final image in output memory, e.g. coordinates, color, depth...



F. Durand, "A Short Introduction to Computer Graphics", MIT Laboratory for Computer Science



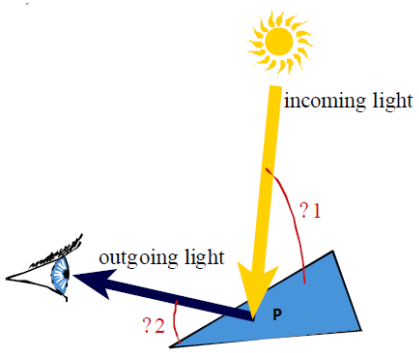
R. Fernando, C. Zeller, "Programming Graphics Hardware Programming Hardware", NVIDIA.

Graphic Pipeline

Several mathematical computation stages to realize 3D virtual scenes into 2D images.

Geometry-Stage:

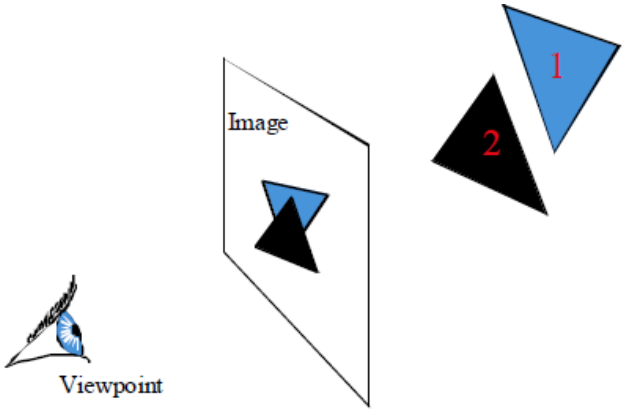
lighting properties of all vertices should be found by considering a light source and a viewer at desired point and their angle.



Lighting Process

Rasterization(光栅化)-Stage:

Drawing covered pixels by the primitives (line or triangle) is called rasterization.

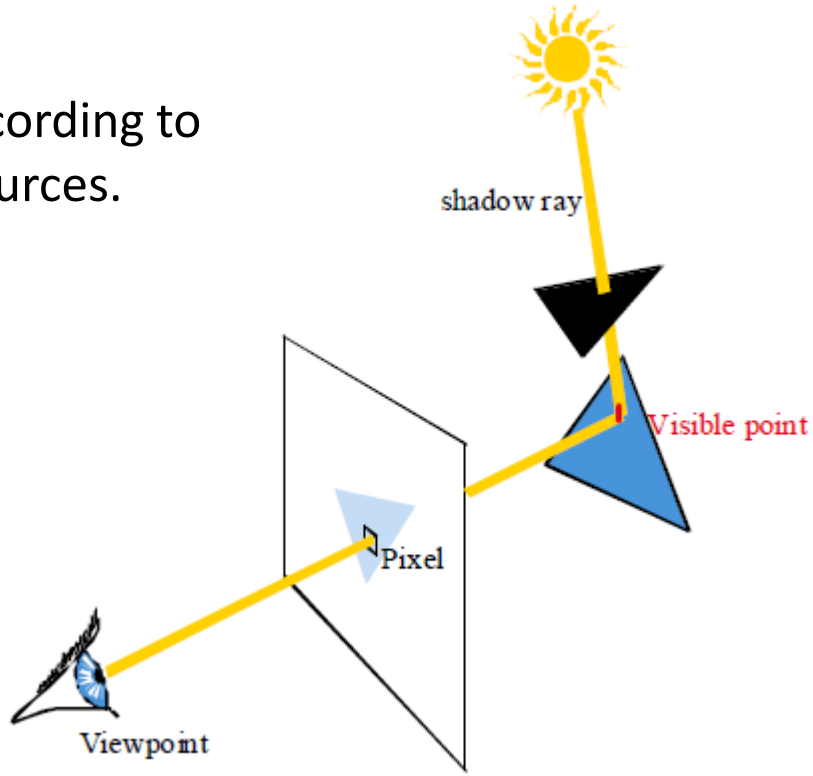


F. Durand, "A Short Introduction to Computer Graphics", MIT Laboratory for Computer Science

Graphic Pipeline

Shading-Stage:

Objects should be shaded according to their interaction with light sources.

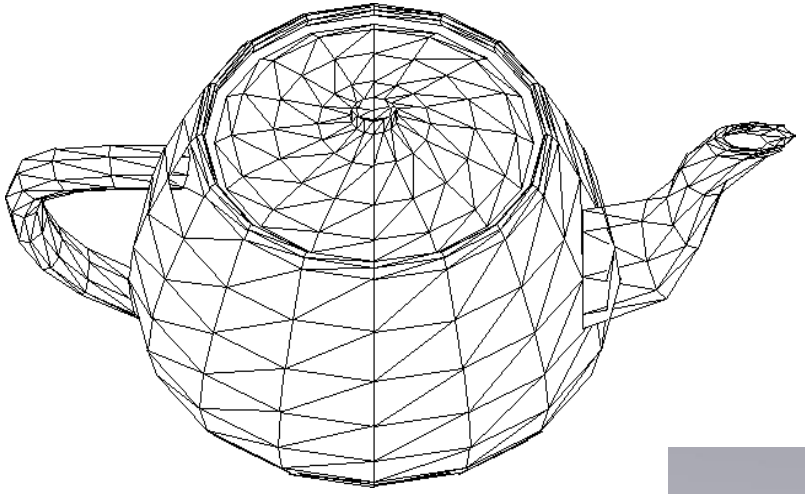


R. Fernando, C. Zeller, "Programming Graphics Hardware Programming Hardware", NVIDIA.

From Numbers to Screen

```
0.748952 -0.764952 -0.210132,
0.672245 -0.690862 -0.210132,
1.00016 -0.369596 -0.210132,
1.00939 -0.004004 -0.210132,
1.14496 0.324436 -0.210132,
1.15747 0.691712 -0.210132,
1.08016 0.793529 -0.210132,
0.98164 0.872032 -0.210132,
0.808263 0.929016 -0.210132,
0.442563 0.985585 -0.210132,
0.221794 1.00159 -0.210132,
0 1.0053 -0.210132,
-0.221794 1.00159 -0.210132,
-0.442563 0.985585 -0.210132,
-0.808263 0.929016 -0.210132,
```





Render triangles.

NVIDIA GTX480 can render 1.6 billion triangles per second!



Before GPUs

- **Vertices to pixels:**
  - Transformations done on **CPU**
  - Compute each pixel “by hand”, in series… **slow!**

Example:

1 million triangles * 100 pixels per triangle *

10 lights * 4 cycles per light computation =

4 billion cycles

Early GPUs: Early 80s to Late 90s

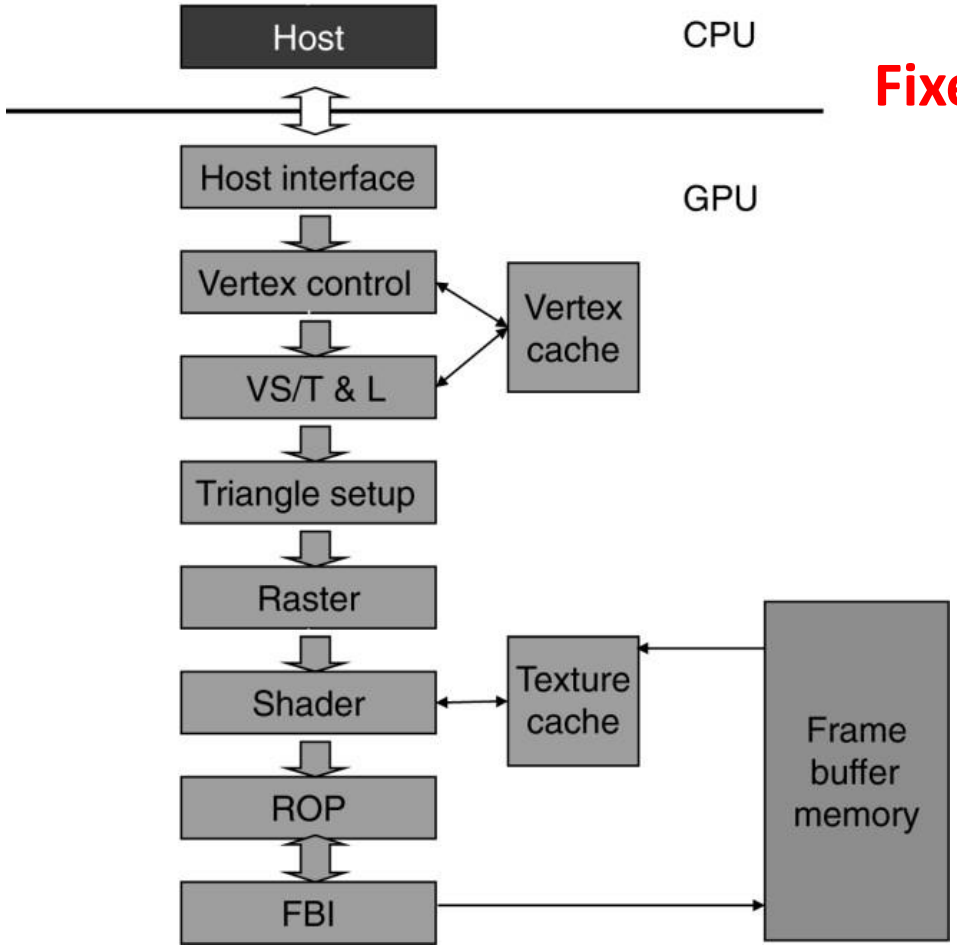
Color Graphic Adapter (CGA 1981)

- 4 bit RGBI
- 40 x 25 Characters
- 320 x 200 Pixel, 16 Colors
- 640 x 200 Pixel Maximum
- 16 Kilo Byte Memory



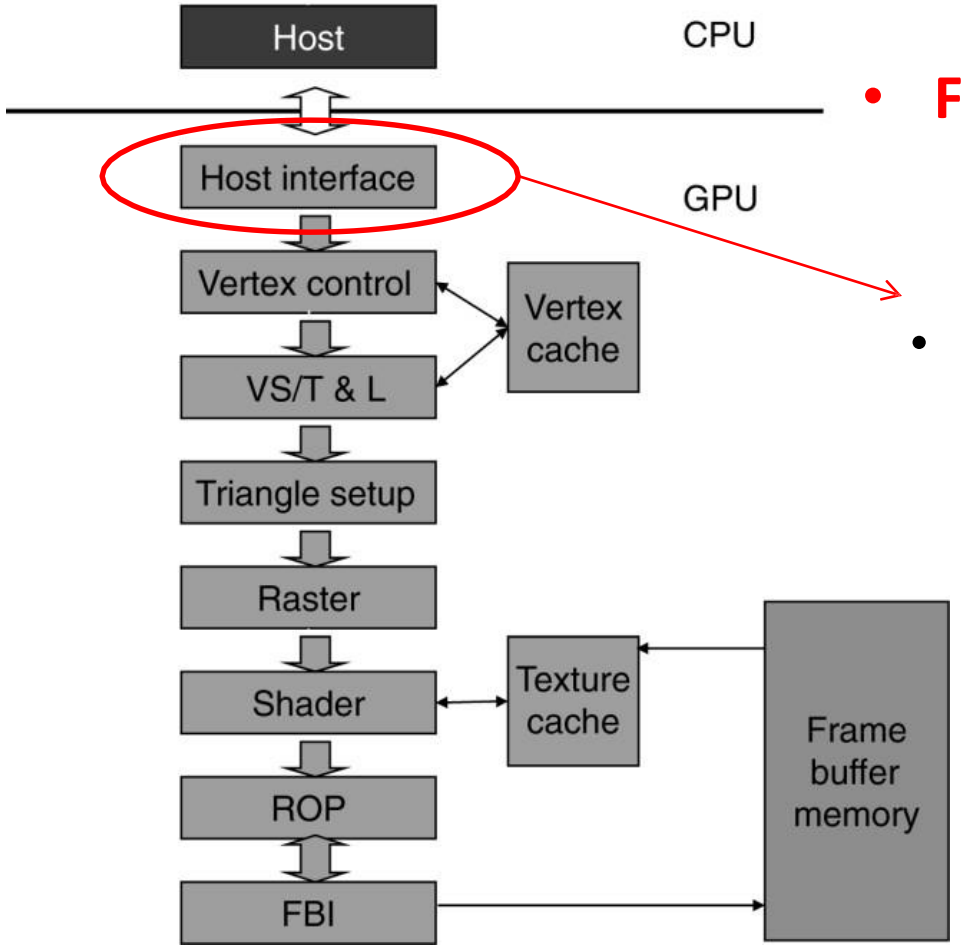


Early GPUs: Early 80s to Late 90s



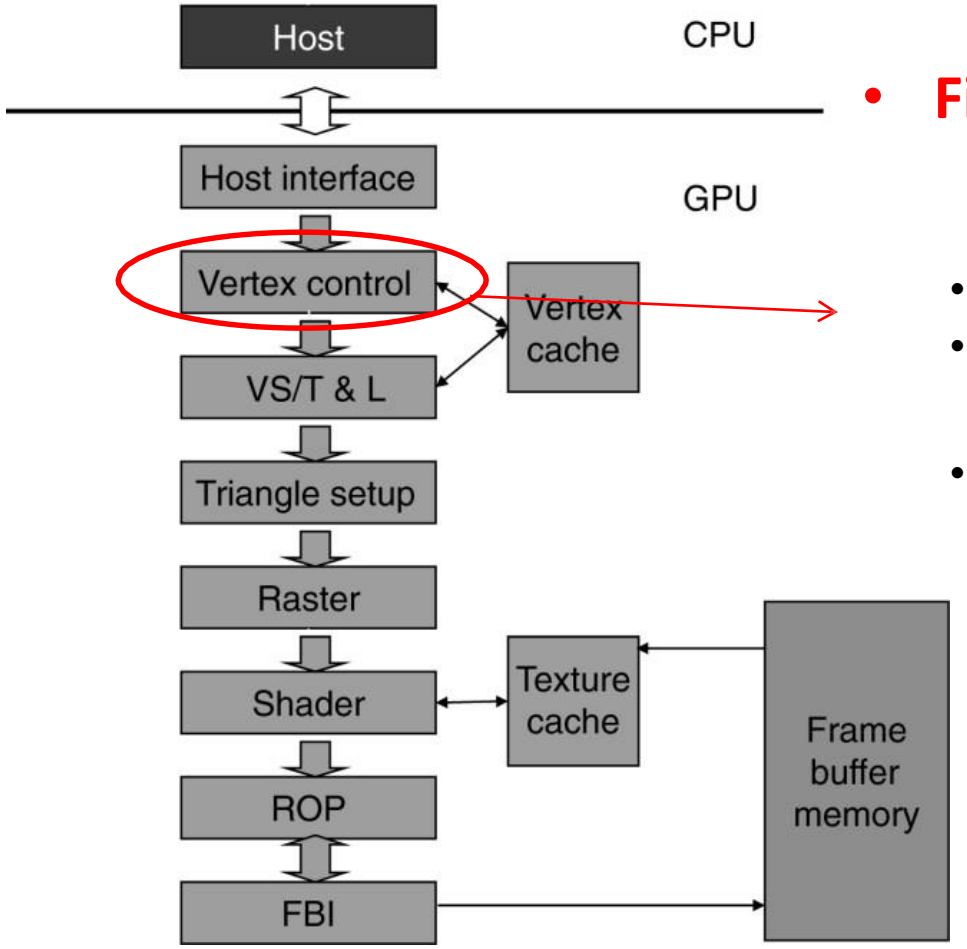
Fixed-Function Pipeline

Early GPUs: Early 80s to Late 90s



- **Fixed-Function Pipeline**
- Receives graphics commands and data from CPU

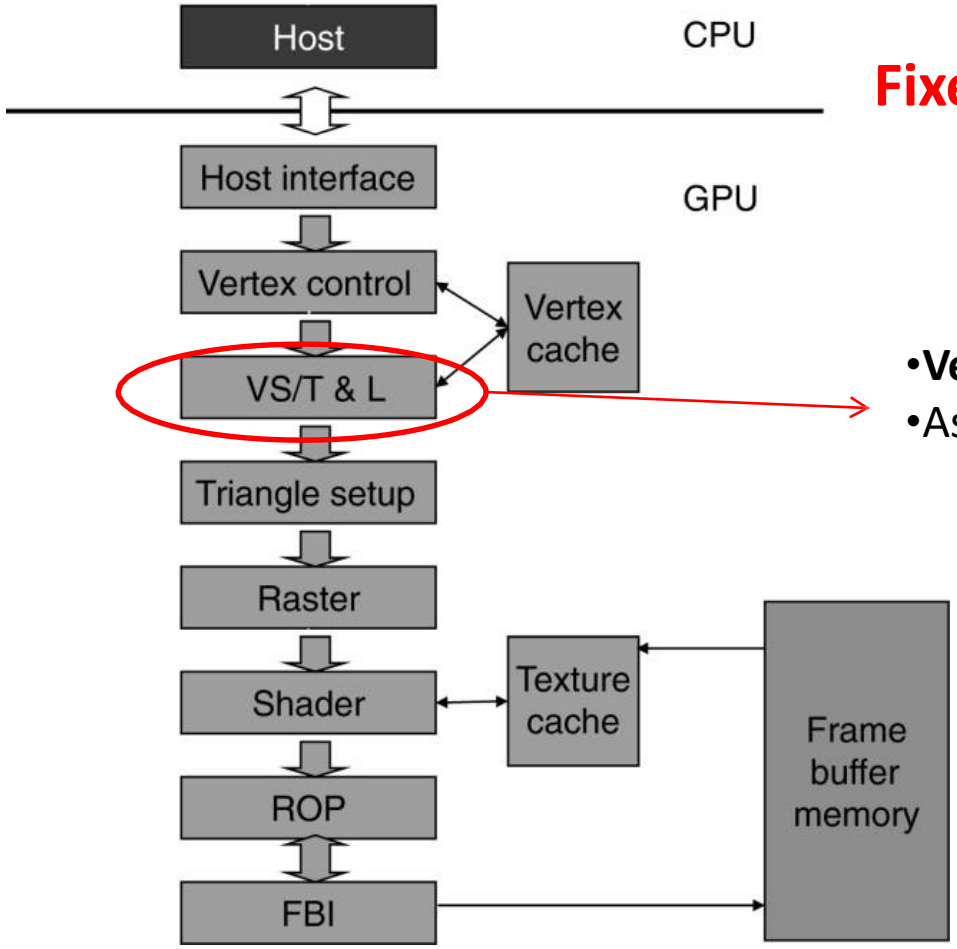
Early GPUs: Early 80s to Late 90s



- **Fixed-Function Pipeline**

- Receives triangle data
- Converts them into a form that hardware understands
- Store the prepared data in vertex cache

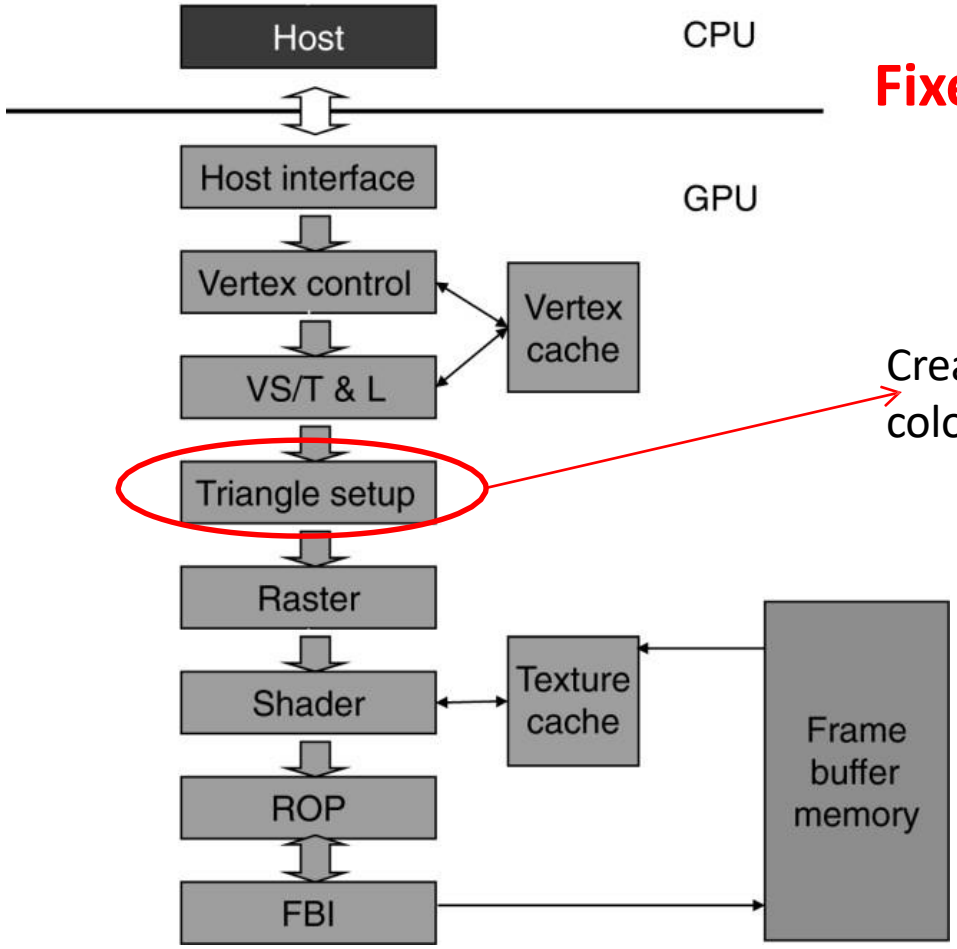
Early GPUs: Early 80s to Late 90s



Fixed-Function Pipeline

- **Vertex shading transform and lighting**
- Assigns per-vertex value (colors, …).

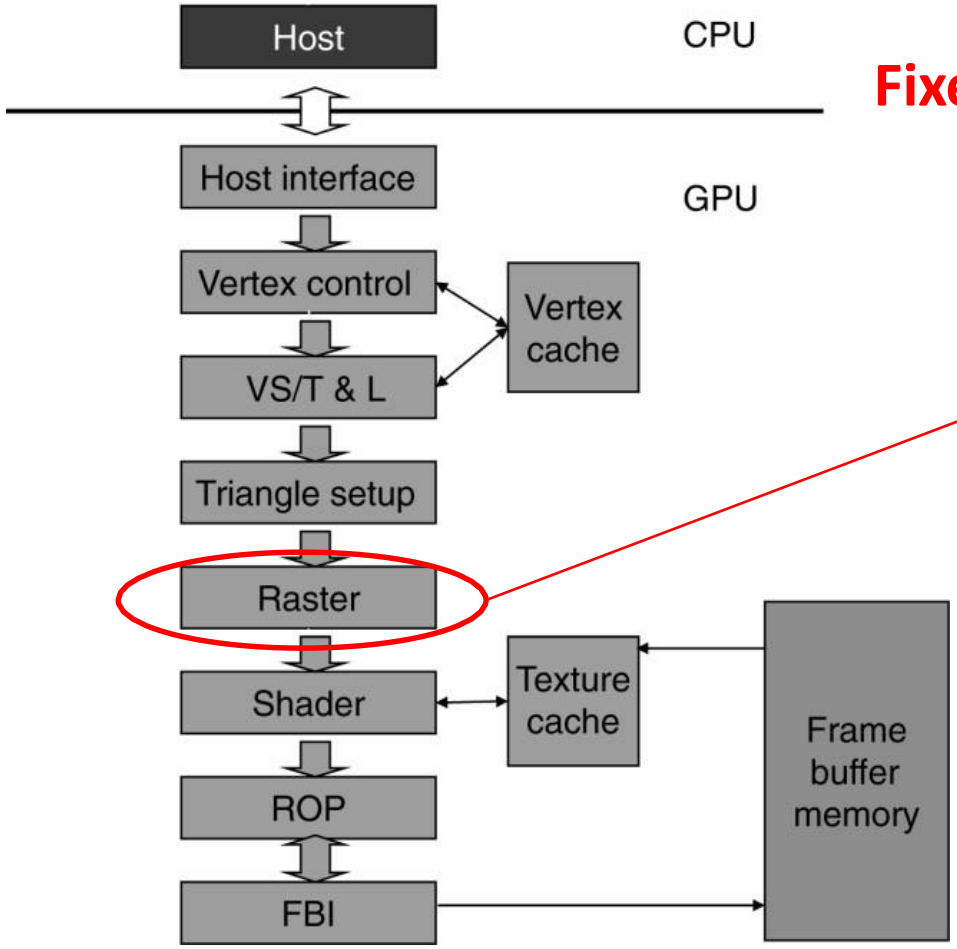
Early GPUs: Early 80s to Late 90s



Fixed-Function Pipeline

Creates edge equations to interpolate colors across pixels touched by the triangle

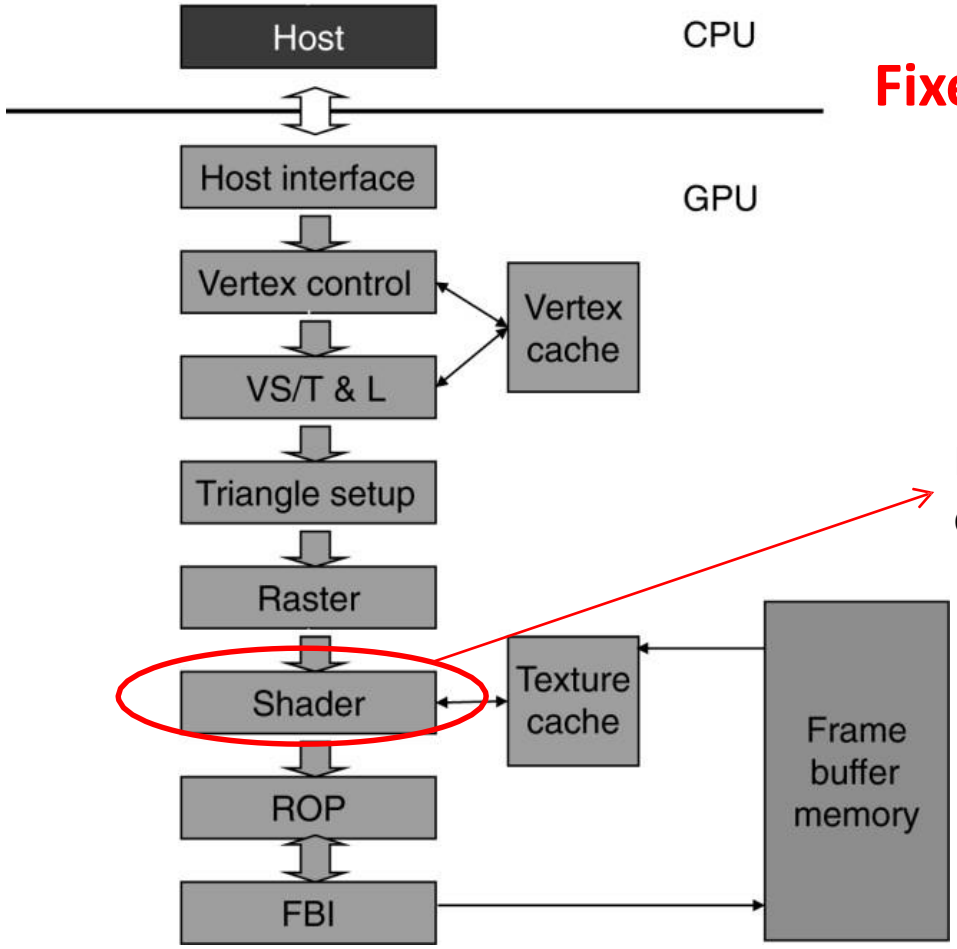
Early GPUs: Early 80s to Late 90s



Fixed-Function Pipeline

- Determines which pixel falls into which triangle
- For each pixel, interpolate per-pixel values from vertices

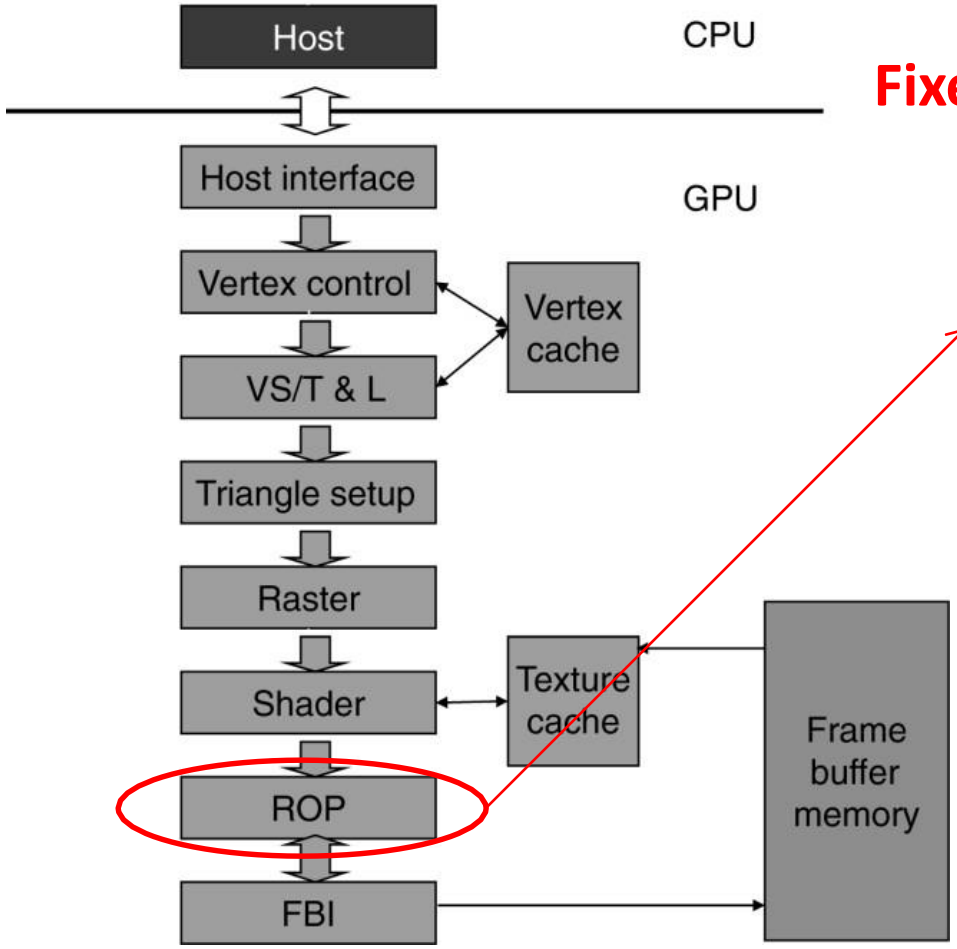
Early GPUs: Early 80s to Late 90s



Fixed-Function Pipeline

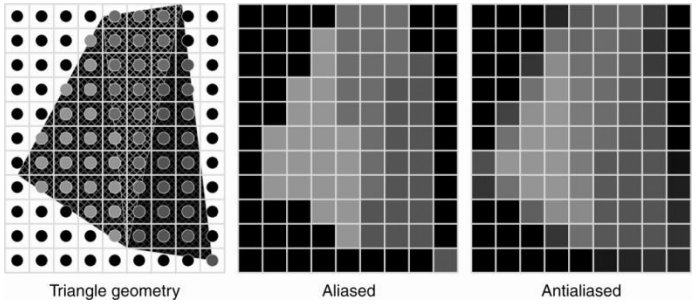
Determines the final color of each pixel

Early GPUs: Early 80s to Late 90s

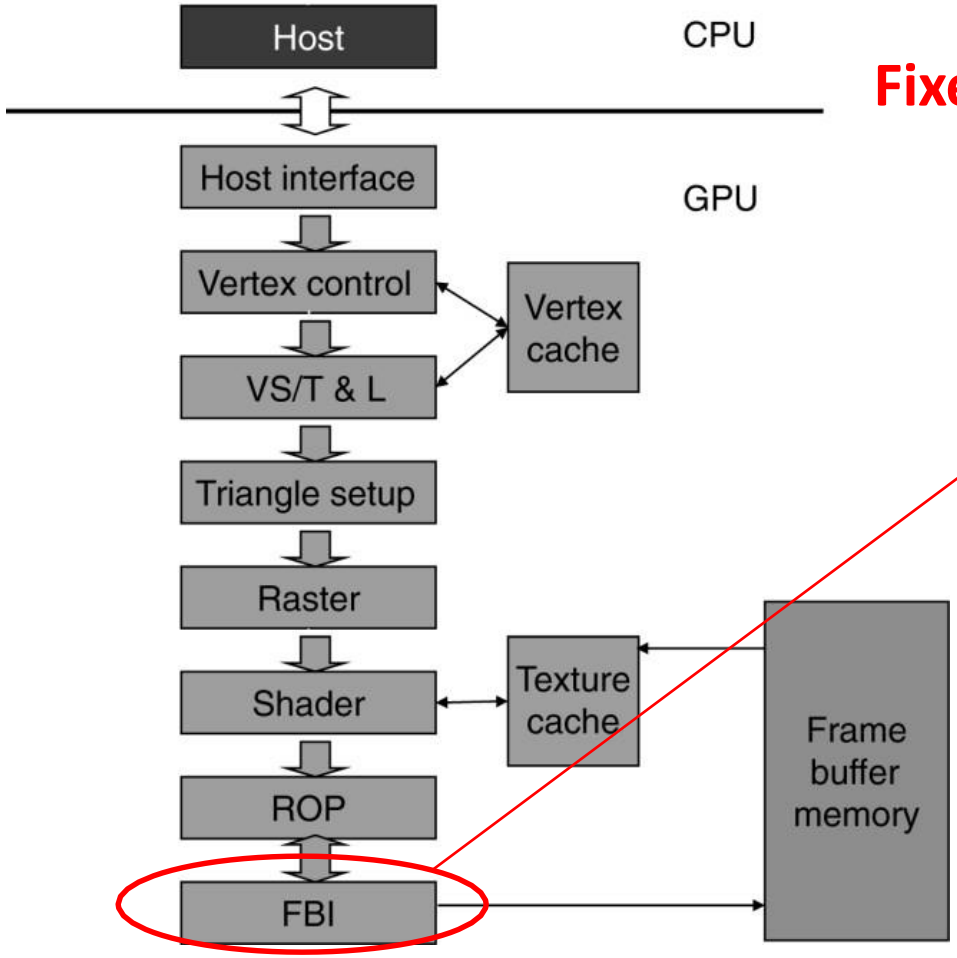


Fixed-Function Pipeline

The **raster operation**:
performs color raster operations that blend the color of overlapping objects for transparency and antialiasing



Early GPUs: Early 80s to Late 90s



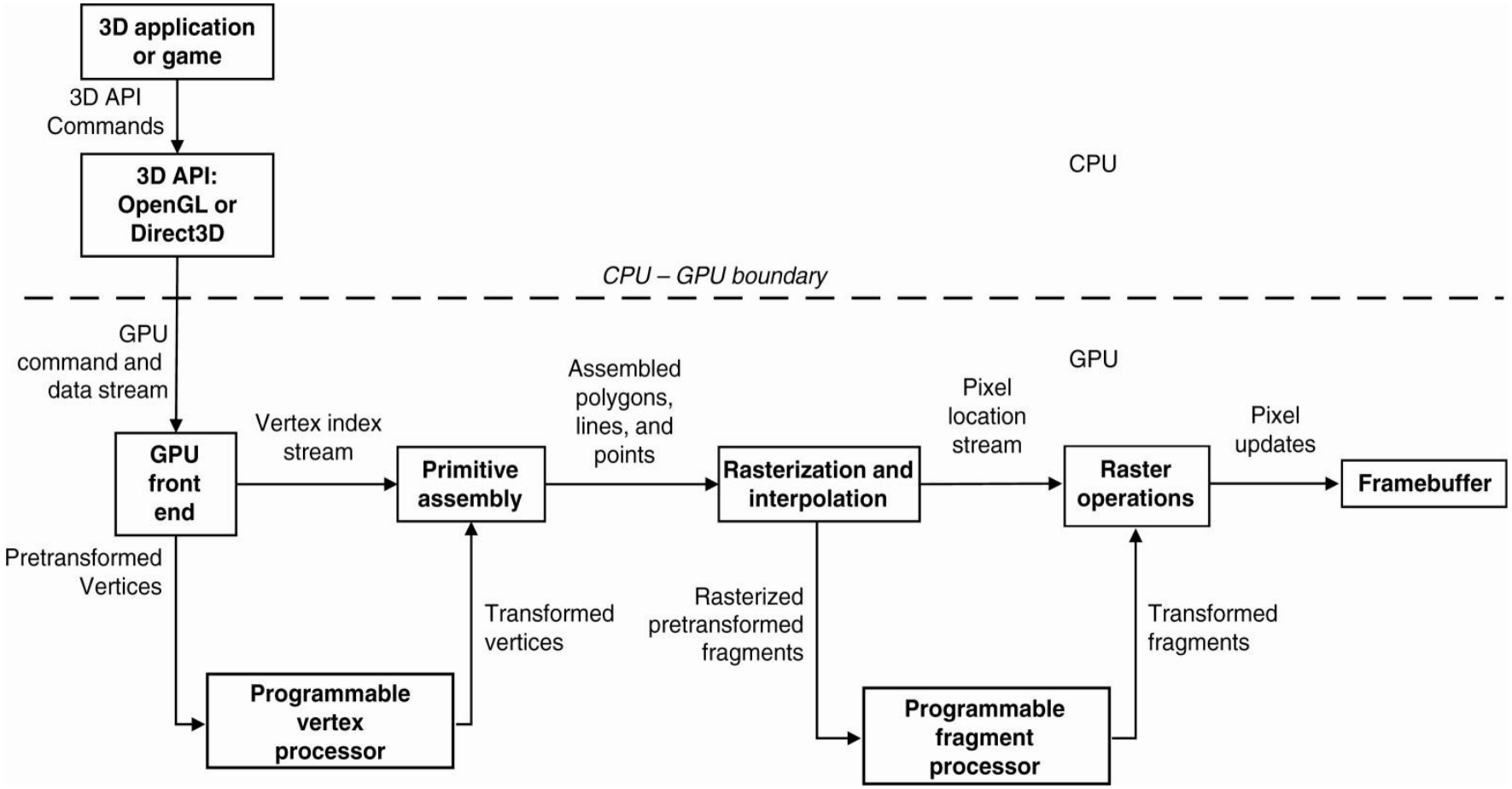
Fixed-Function Pipeline

The **frame buffer interface** manages memory reads/writes.

Next Steps

- In 2001:
  - NVIDIA exposed the application developer to the instruction set of VS/T&L stage

- Later:
  - **General programmability** extended to shader stage → trend toward unifying the functionality of different stages as seen by application programmer.
  - In graphics pipelines, certain stages do a great deal of floating-points arithmetic on a completely independent data.
    - Data independence is exploited → key assumption in GPUs



Fragment = a technical term usually meaning a single pixel

The Vision of NVIDIA

- "Within the next few years, there will be single-chip graphics devices more powerful and versatile than any graphics system that has ever been built, at any price."



-- David Kirk, NVIDIA, 1998

In 2006

- NVIDIA GeForce 8800 mapped separate graphics stage to a unified array of processors
  - For vertex shading, geometry processing, and pixel processing
  - Allows dynamic partition

Regularity + Massive Parallelism









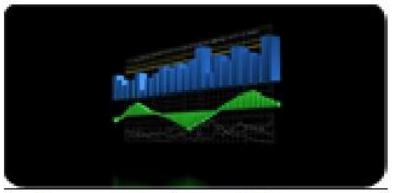

General Purposed GPU



Bio-Informatics and Life Sciences



Computational Electromagnetics and Electrodynamics



Computational Finance



Computational Fluid Dynamics



Data Mining, Analytics, and Databases



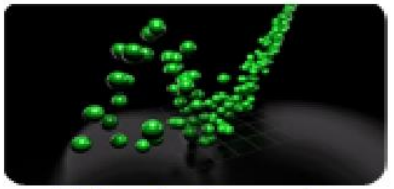
Imaging and Computer Vision



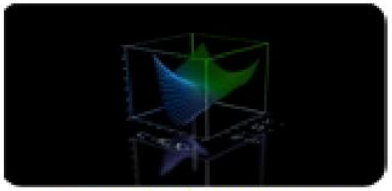
MATLAB Acceleration



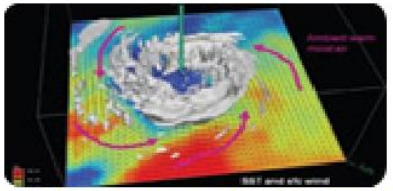
Medical Imaging



Molecular Dynamics

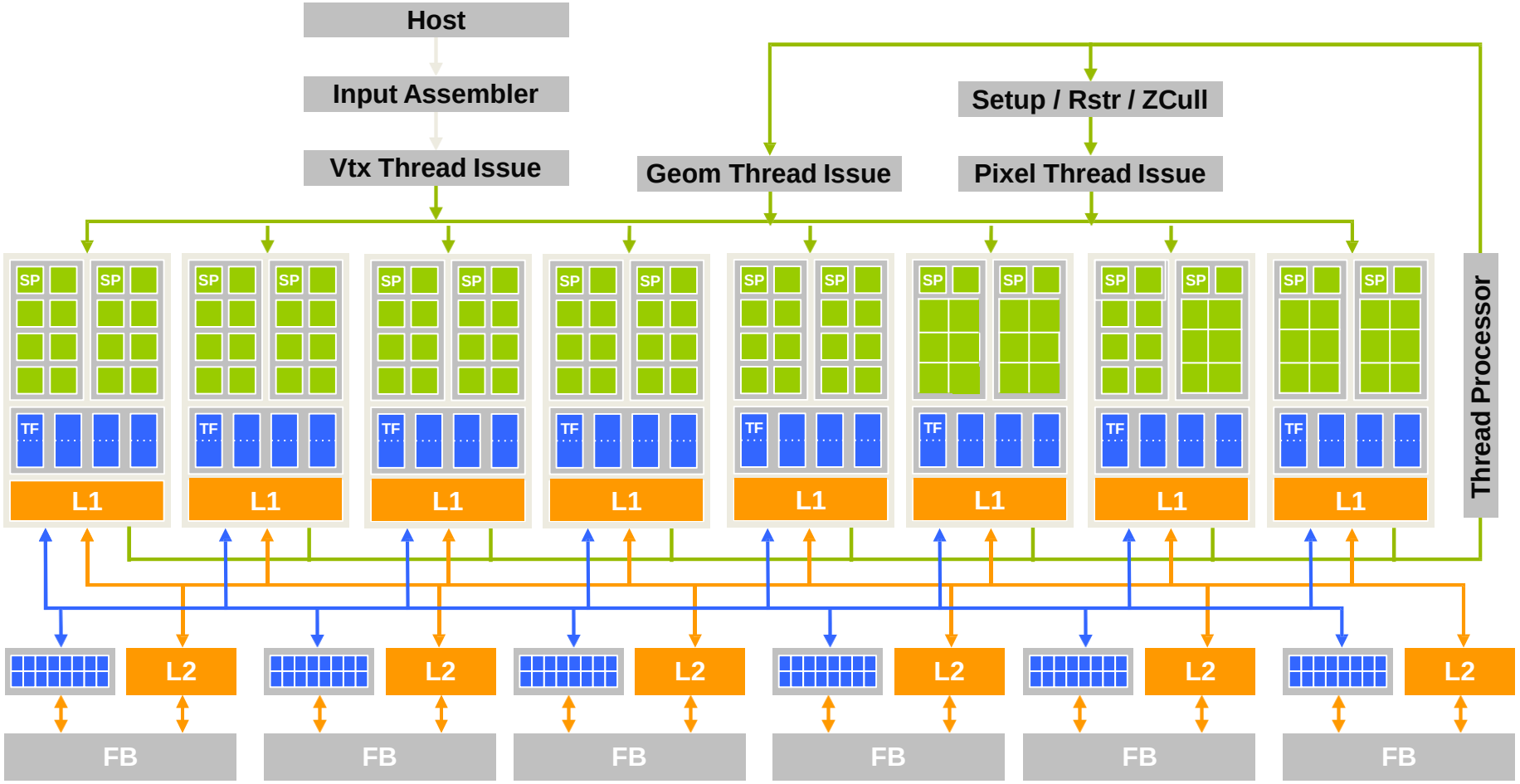


Numerical Packages



Weather, Atmospheric, Ocean Modeling, and Space Sciences

ref: http://www.nvidia.com/object/tesla_computing_solutions.html



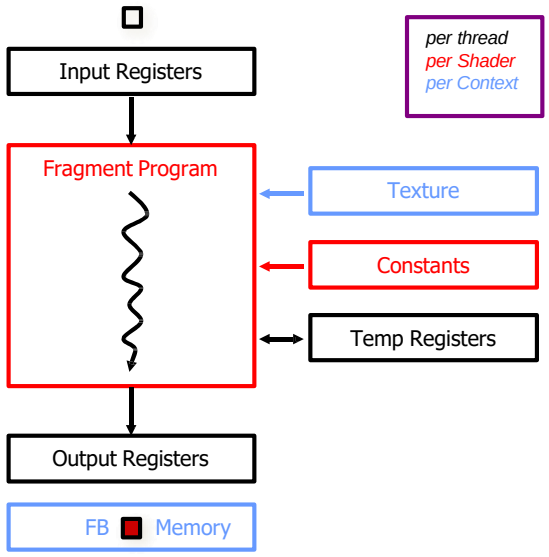
Exploring the use of GPUs to solve *compute intensive* problems

The birth of GPGPU but there are many constraints

GPUs and associated APIs were designed to process graphics data

Previous GPGPU Constraints

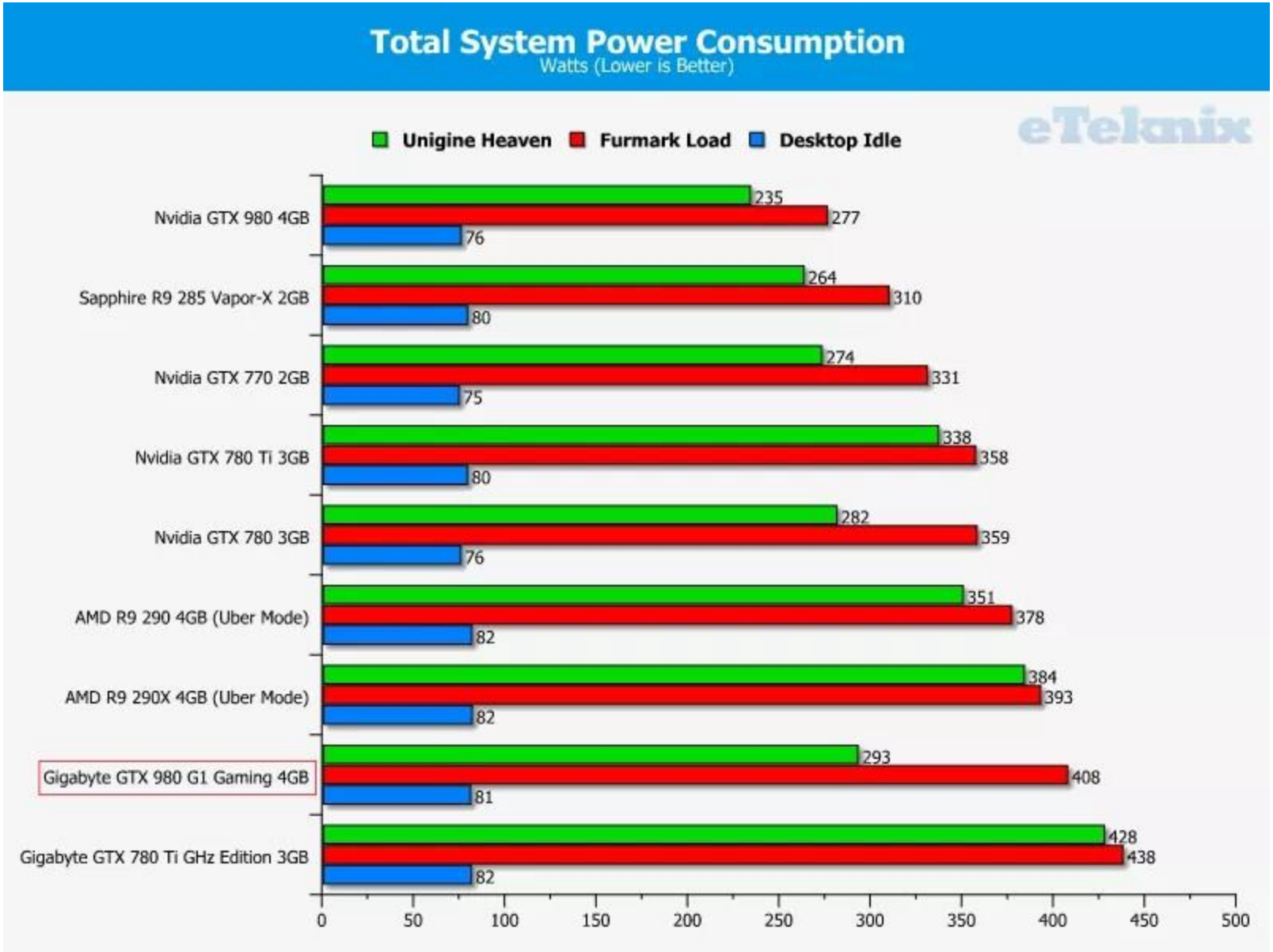
- **Dealing with graphics API**
  - Working with the corner cases of the graphics API
- **Addressing modes**
  - Limited texture size/dimension
- **Shader capabilities**
  - Limited outputs
- **Instruction sets**
  - Lack of Integer & bit ops
- **Communication limited**
- **No user-defined data types**



The Birth of GPU Computing

- **Step 1**: Designing high-efficiency floating-point and integer processors.
- **Step 2**: Exploiting data parallelism by having large number of processors
- **Step 3**: Shader processors fully programmable with large instruction cache, instruction memory, and instruction control logic.
- **Step 4**: Reducing the cost of hardware by having multiple shader processors to share their cache and control logic.
- **Step 5**: Adding memory load/store instructions with random byte addressing capability
- **Step 6**: Developing CUDA C/C++ compiler, libraries, and runtime software models.

Problem With GPUs: Power



Source: http://www.eteknix.com/gigabyte-g1-gaming-geforce-gtx-980-4gb-graphics-card-review/17/

Problems Faced by GPUs

- Need enough parallelism
- Under-utilization
- Bandwidth to CPU

Still a way to go

Modern GPU Hardware

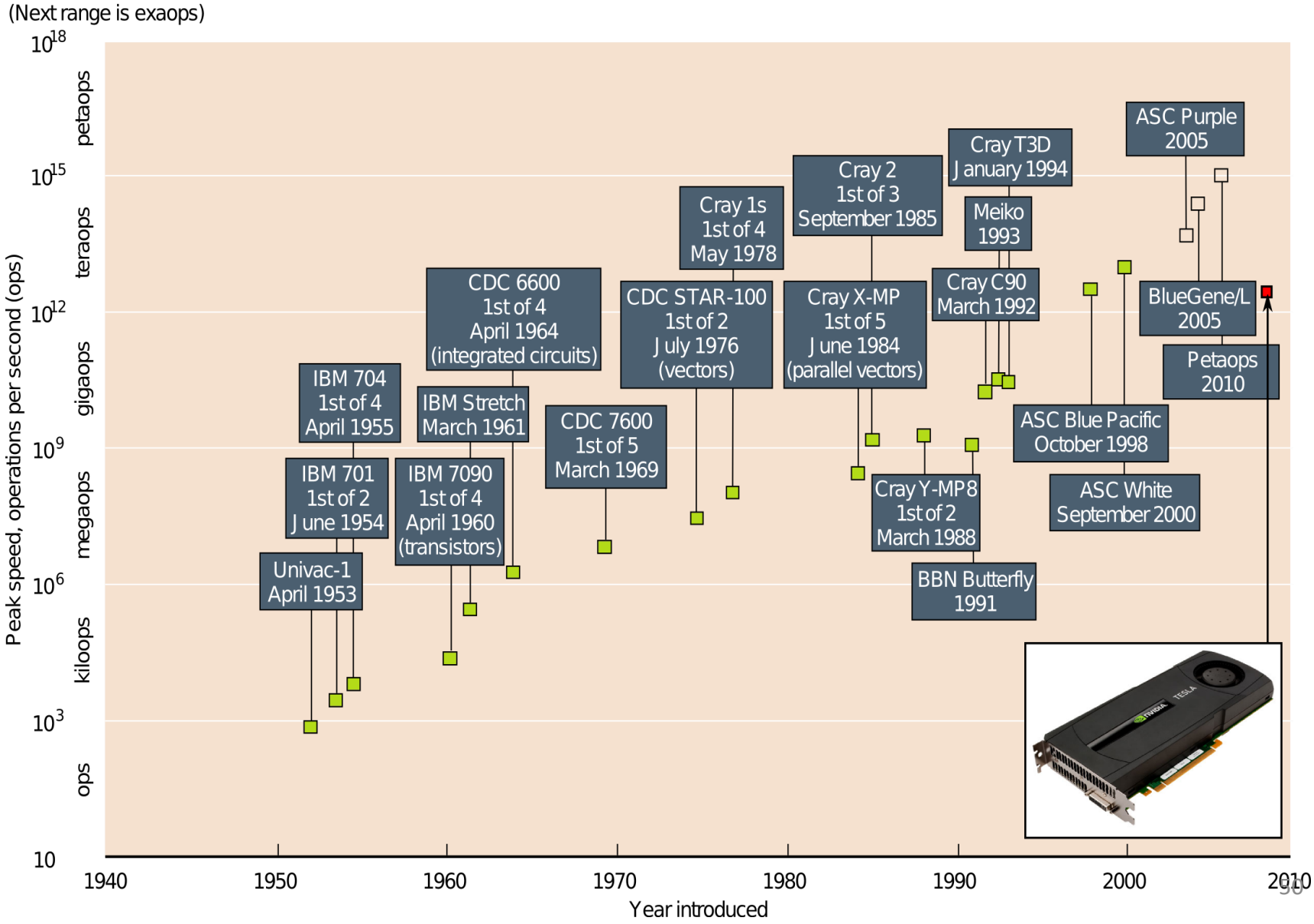
Modern GPU Hardware

- GPUs have many parallel execution units (called cores in general, and SP or CUDA cores in NVIDIA lingo) and higher transistor counts

- GPUs have much deeper pipelines
  - several hundreds stages vs 10-20 for CPUs

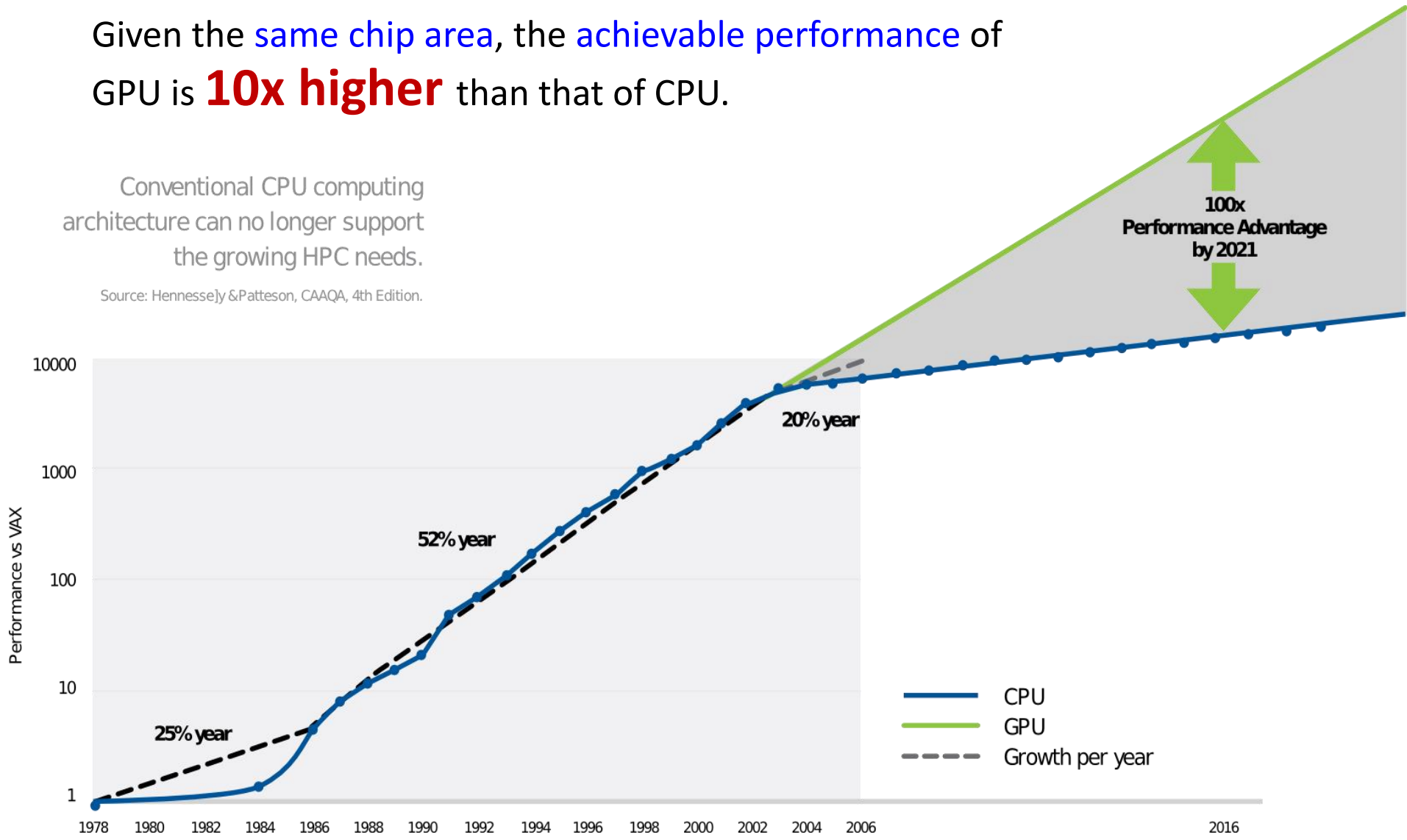
- GPUs have significantly faster and more advanced memory interfaces

Single-Chip GPU vs Fastest Supercomputers



The Gap Between CPU and GPU

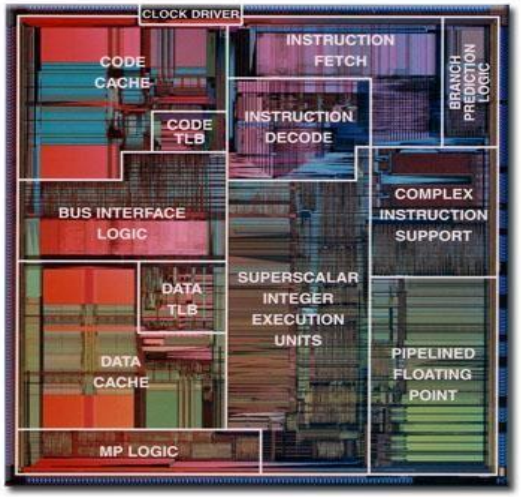
Given the same chip area, the achievable performance of GPU is **10x higher** than that of CPU.



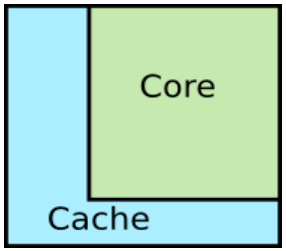
ref: Tesla GPU Computing Brochure

Evolution of Intel Pentium

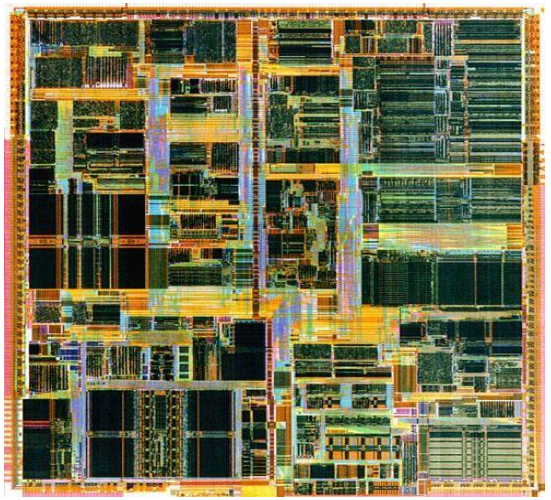
Pentium I

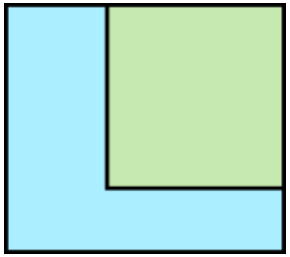


Chip area breakdown

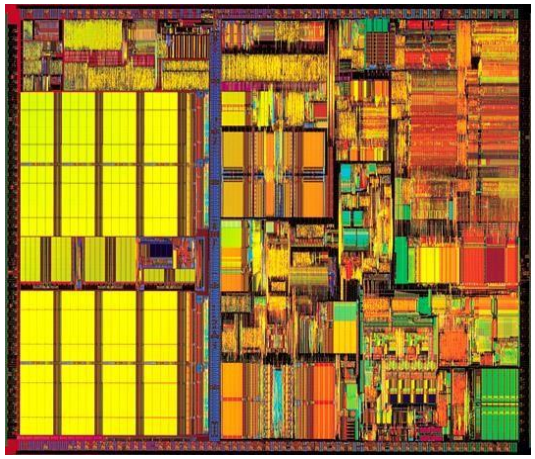


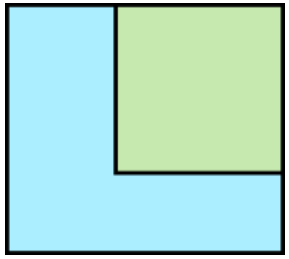
Pentium II



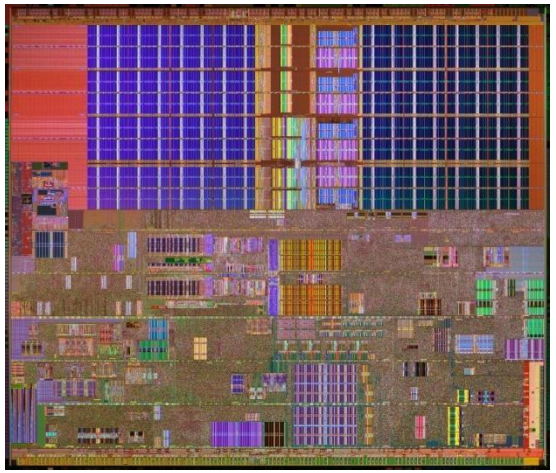


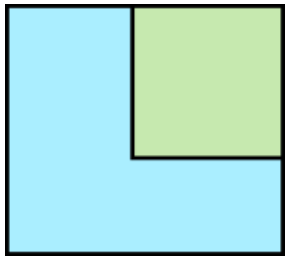
Pentium III





Pentium IV

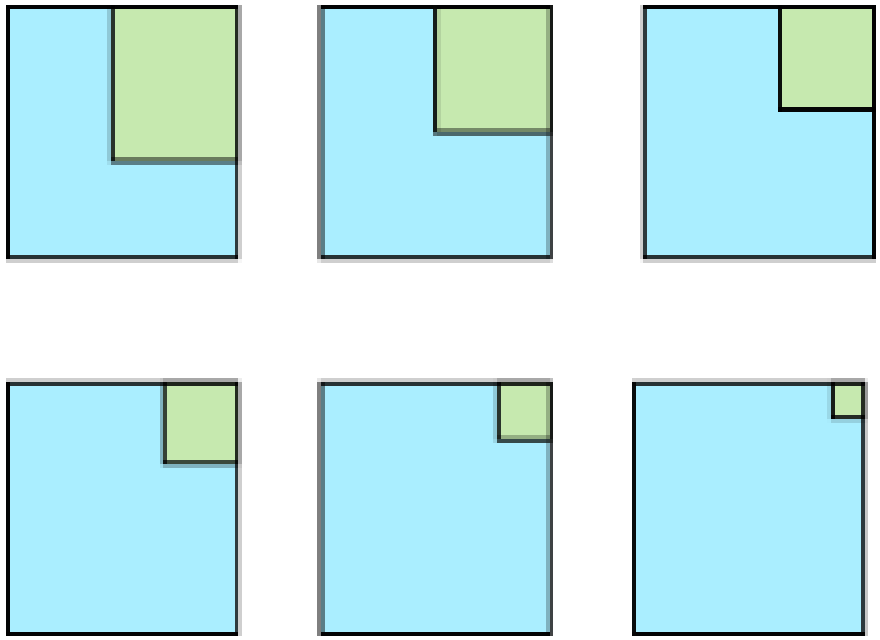




source: http://www.es.ele.tue.nl/~heco/courses/EmbeddedComputerArchitecture/GPU.ppt

Extrapolation of Single Core CPU

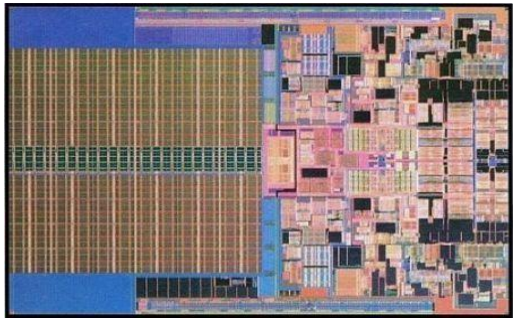
If we extrapolate (推断) the trend, in a few generations, Pentium will look like:



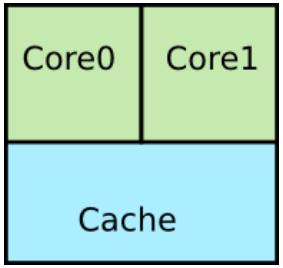
source: http://www.es.ele.tue.nl/~heco/courses/EmbeddedComputerArchitecture/GPU.ppt

Evolution of Multi-core CPUs

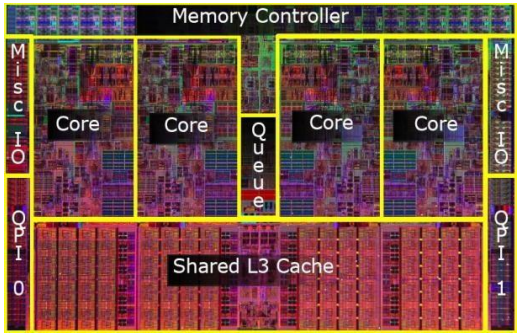
Penryn

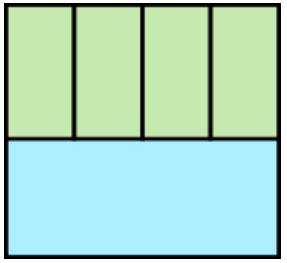


Chip area breakdown

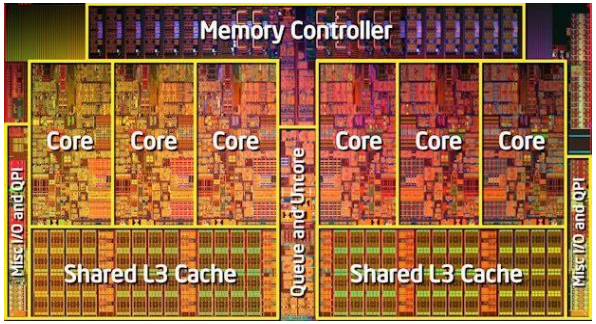


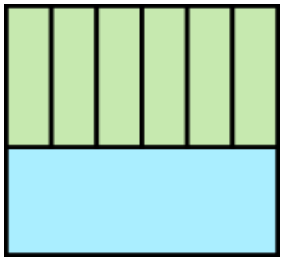
Bloomfield



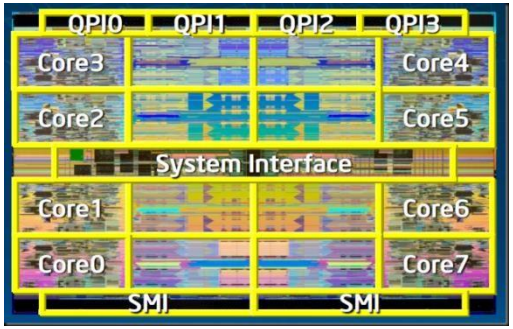


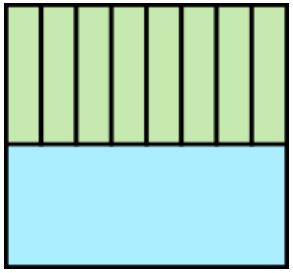
Gulftown





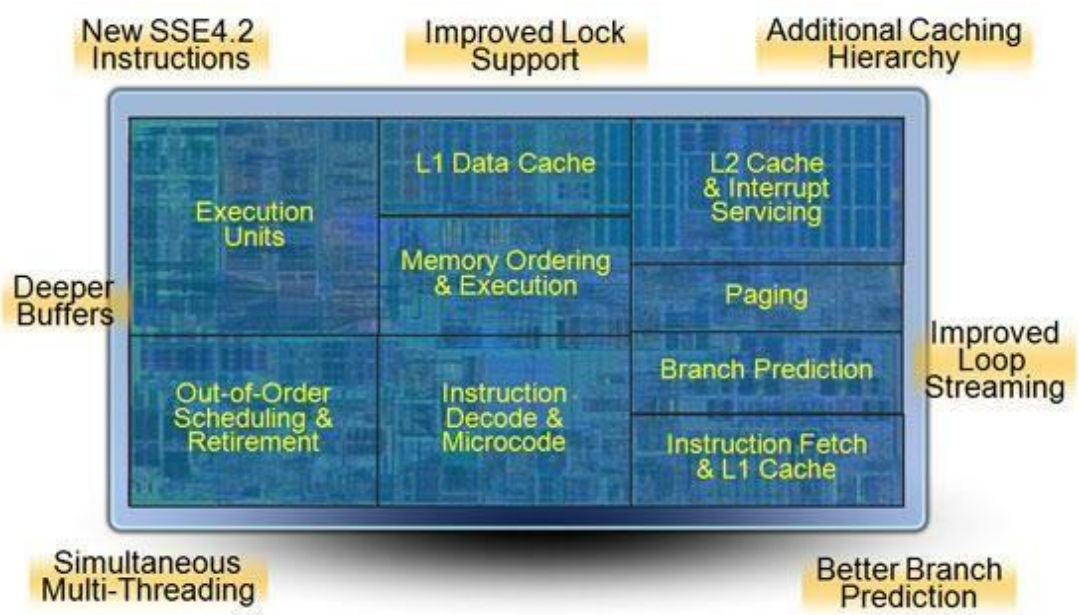
Beckton

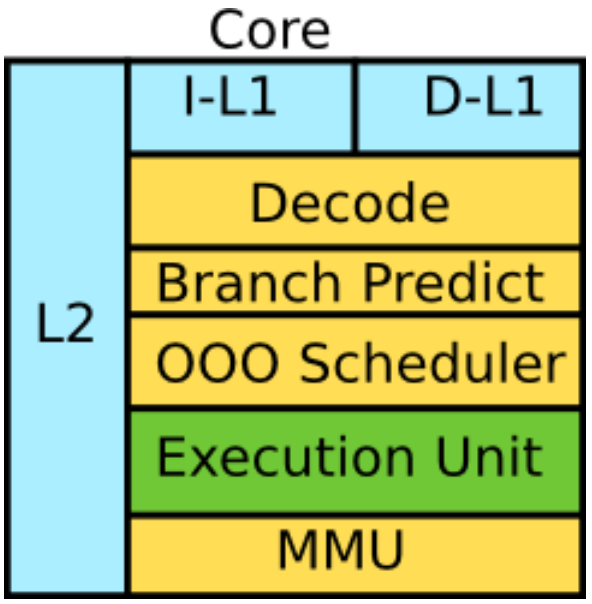




source: http://www.es.ele.tue.nl/~heco/courses/EmbeddedComputerArchitecture/GPU.ppt

Let's Take a Closer Look

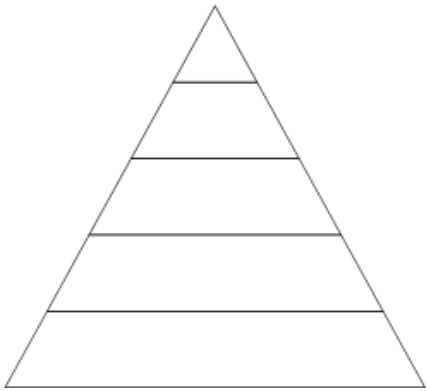




Less than 10% of total chip area is used for the real execution.

source: http://www.es.ele.tue.nl/~heco/courses/EmbeddedComputerArchitecture/GPU.ppt

The Memory Hierarchy



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

Notes on Energy at 45nm:
64-bit Int ADD takes about 1 pJ.
64-bit FP FMA takes about 200 pJ.

It seems we can not further increase the computational density.

The Brick Wall -- UC Berkeley's View

Power Wall: power expensive, transistors free

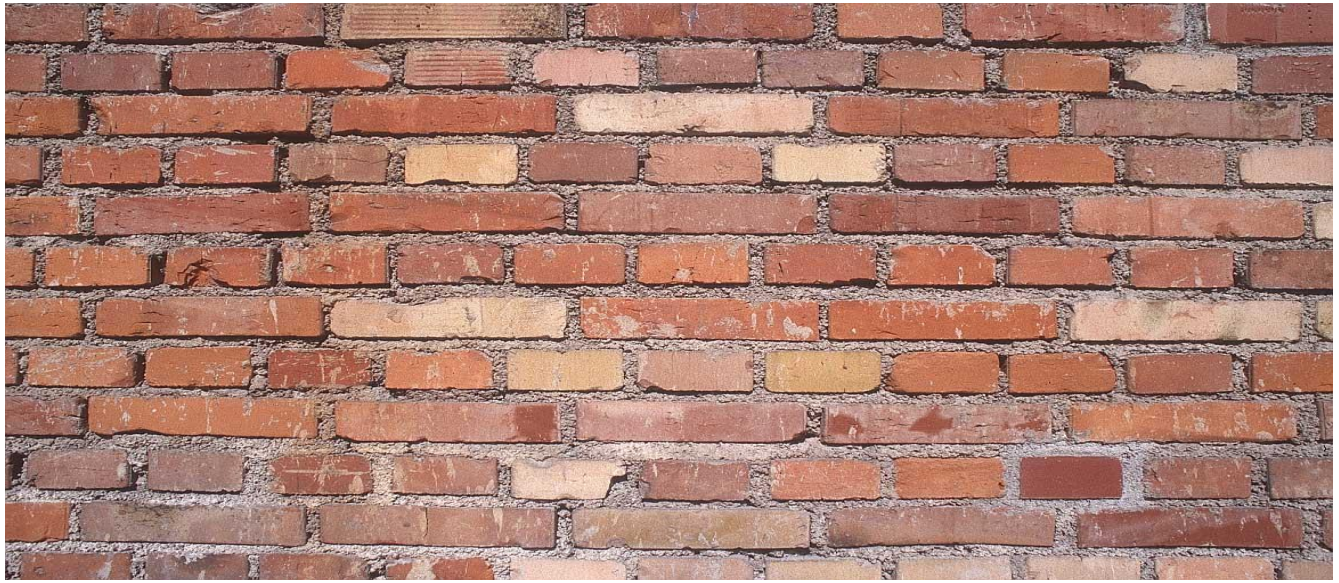
- Can put more on chip than can afford to turn on

Memory Wall: Memory slow, multiplies fast

- 200 clock cycles to DRAM memory, 4 clocks for multiply

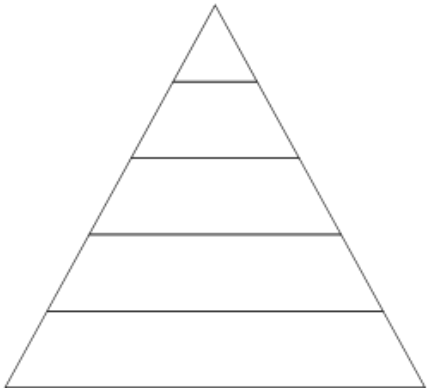
ILP Wall: diminishing returns on more ILP HW

Power Wall + Memory Wall + ILP Wall = Brick Wall



David Patterson, "Computer Architecture is Back - The Berkeley View of the Parallel Computing Research Landscape", Stanford EE Computer Systems Colloquium, Jan 2007, link

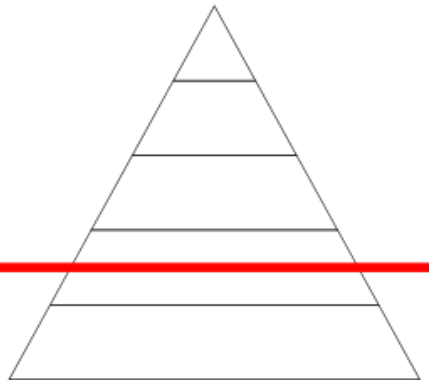
How to Break the Brick Wall?



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

Hint: how to exploit the parallelism inside the application?

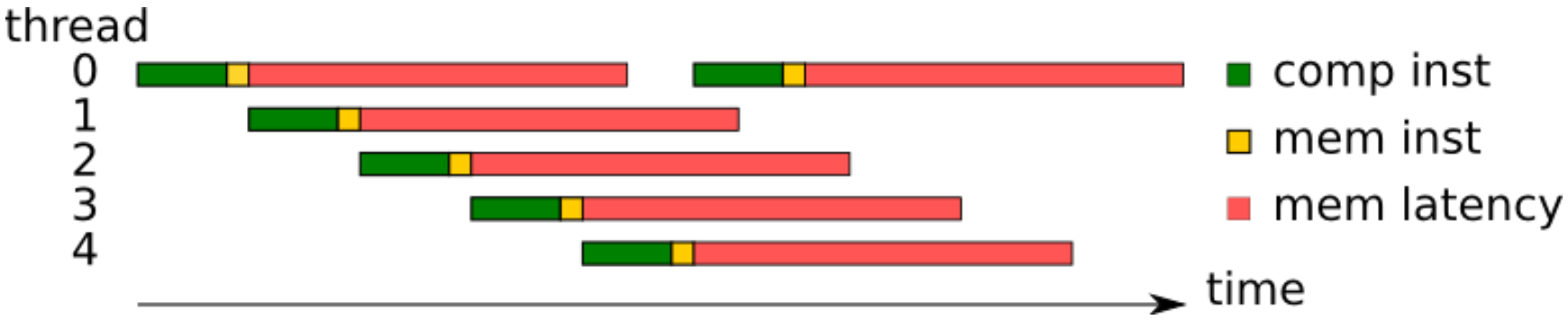
Step 1: Trade Latency with Throughput



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	~~1K~~ 16K	~~10~~ 20	~~1~~ 2~3	1000
L1	32K	20	5	100
L2	256K	100	10	100
~~L3~~	~~8M~~	~~200~~	~~50~~	~~100~~
Off-chip	4G	2000	100	10

Hide the memory latency through fine-grained interleaved (交叉的) threading.

Interleaved Multi-threading

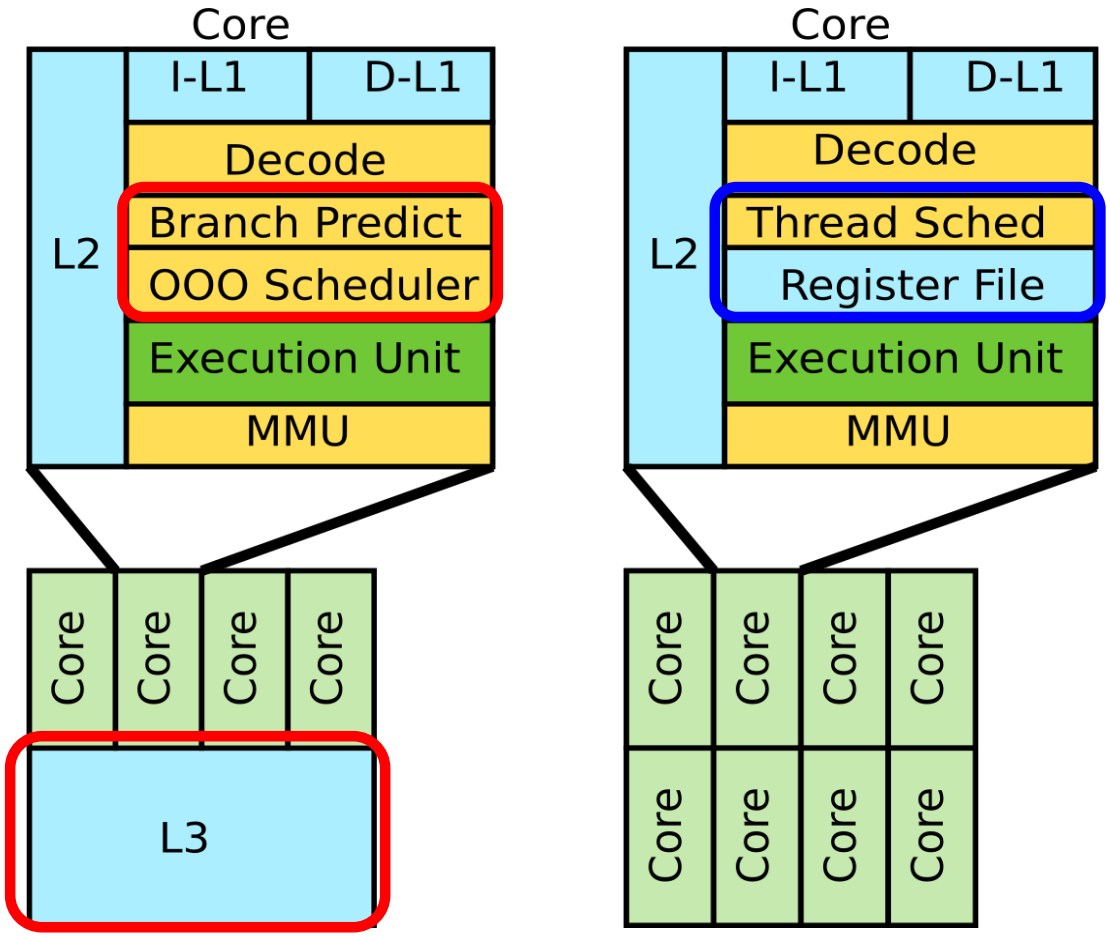


The granularity of interleaved multi-threading:

- 100 cycles: hide off-chip memory latency
- 10 cycles: + hide cache latency
- 1 cycle: + hide branch latency, instruction dependency

Fine-grained interleaved multi-threading:
Pros: remove branch predictor, OOO scheduler, large cache
Cons: register pressure, etc.

Fine-Grained Interleaved Threading



Pros:

reduce cache size,
no branch predictor,
no OOO scheduler

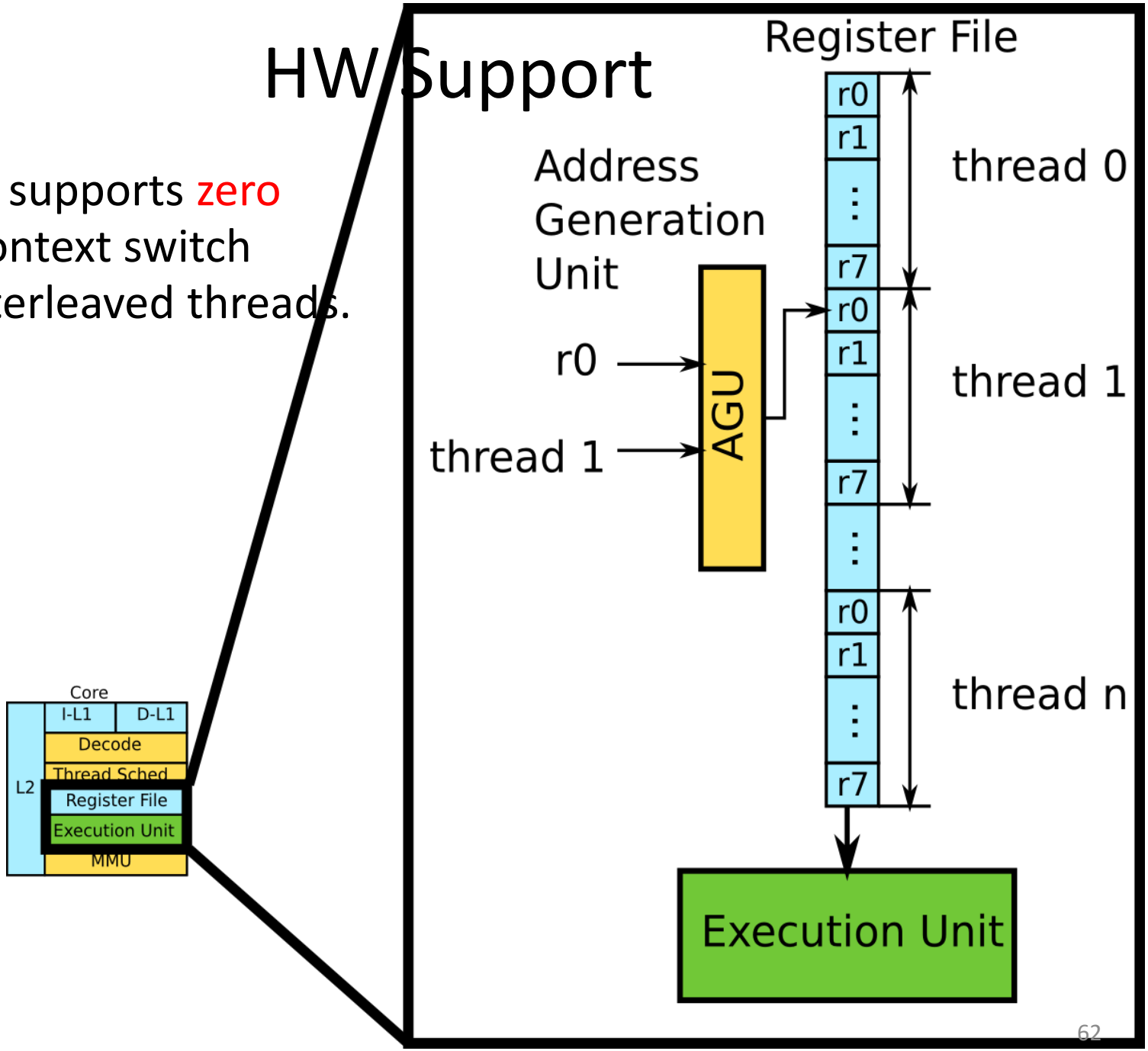
Cons:

register pressure,
thread scheduler,
require huge parallelism

Without and with fine-grained interleaved threading

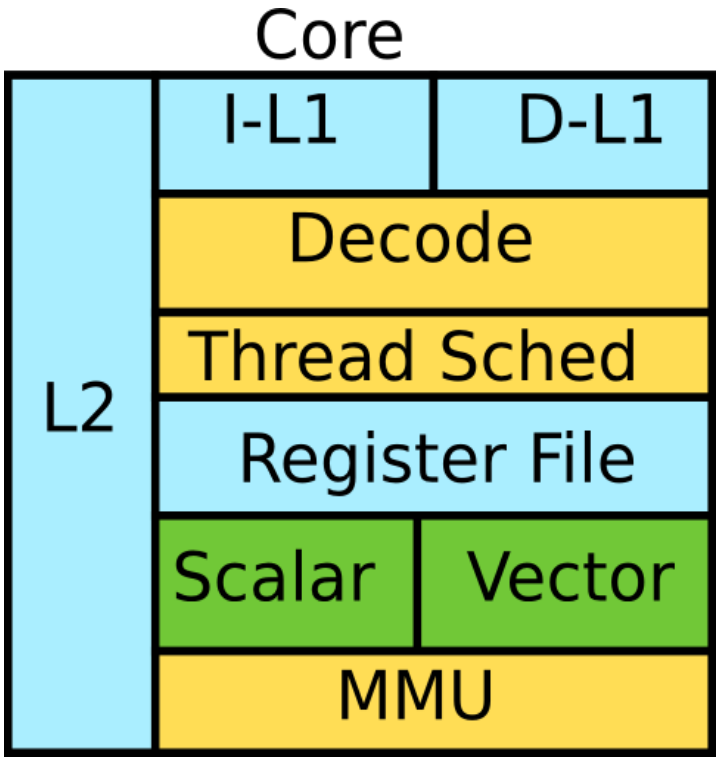
HW Support

Register file supports zero overhead context switch between interleaved threads.



Can We Make Further Improvement?

- Reducing large cache gives 2x computational density.
- **Q: Can we make further improvements?**

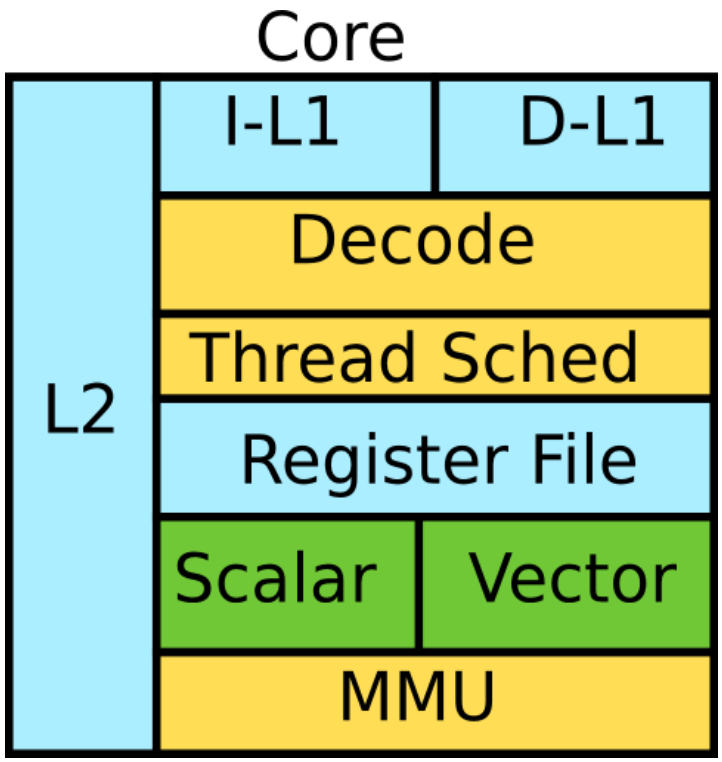


Hint: We have only utilized thread level parallelism (TLP) so far.

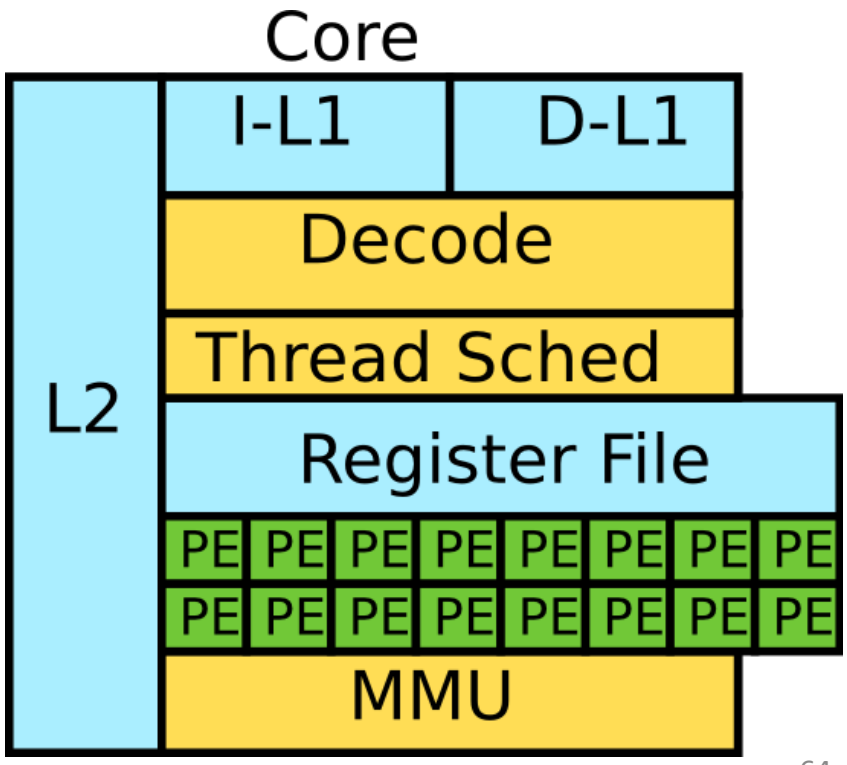
Step 2: Single Instruction Multiple Data

GPU uses wide SIMD: 8/16/24/... processing elements (PEs)
CPU uses short SIMD: usually has vector width of 4.

SSE has 4 data lanes

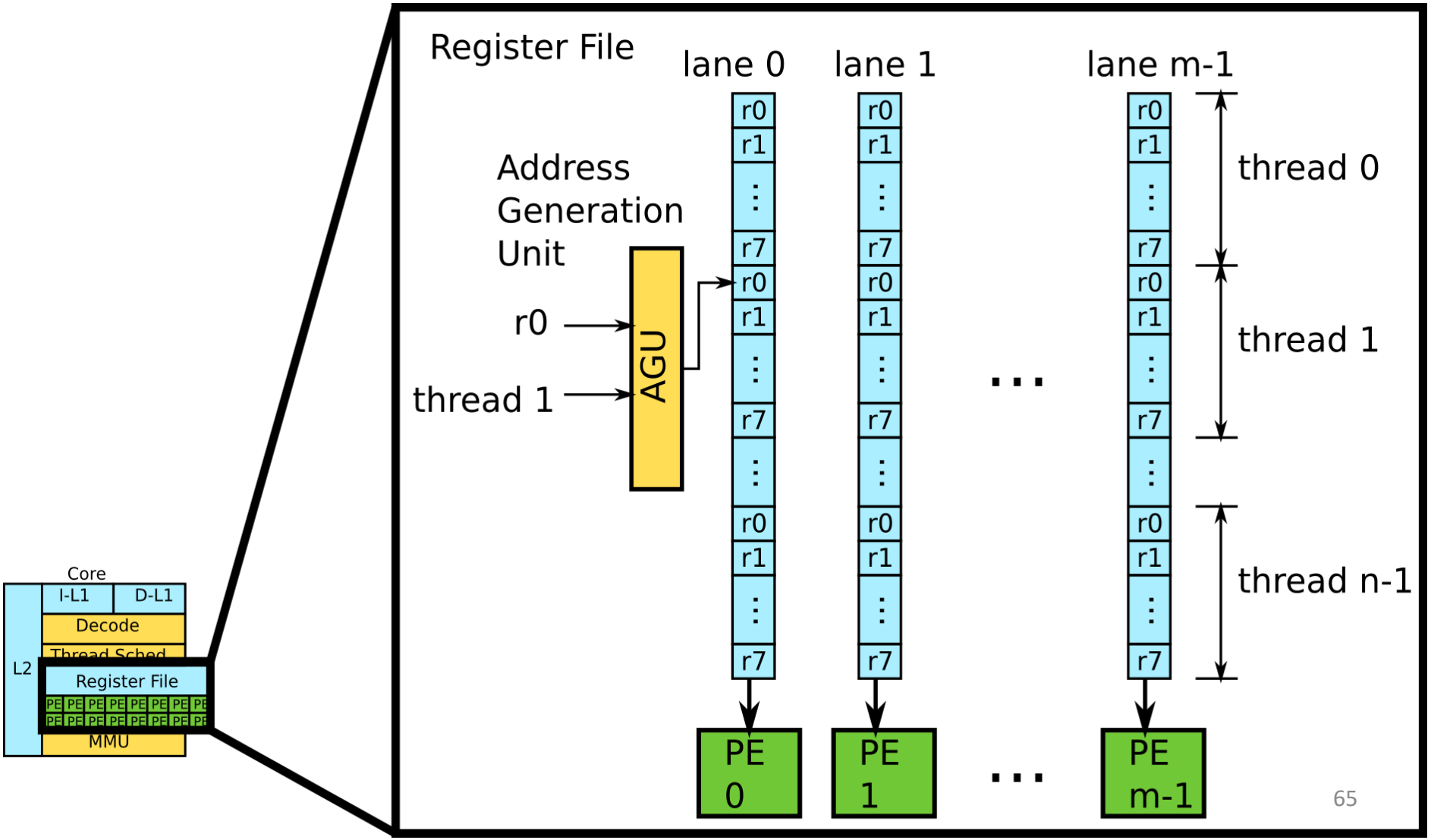


GPU has 8/16/24/... data lanes



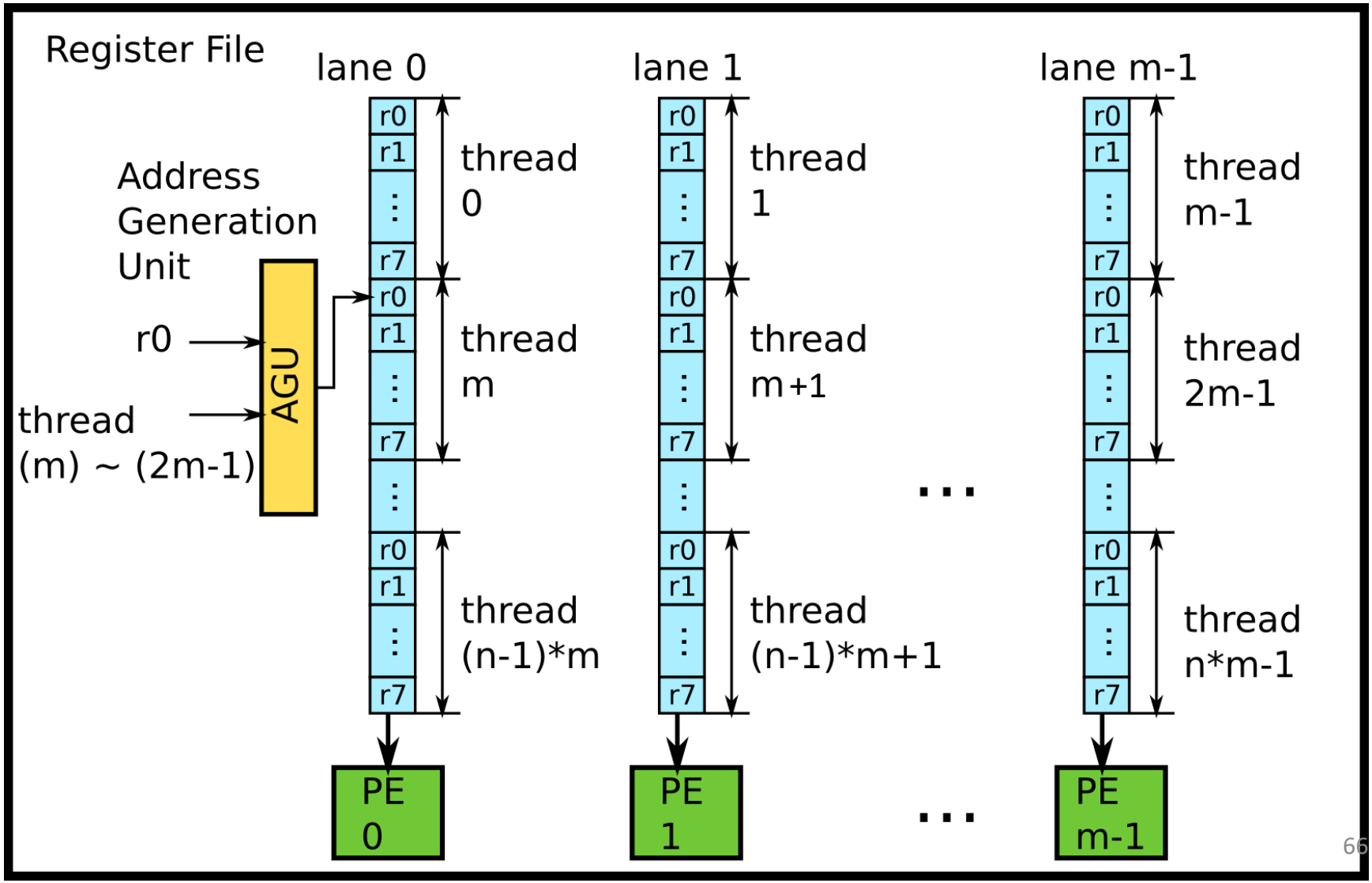
Hardware Support

Supporting interleaved threading + SIMD execution



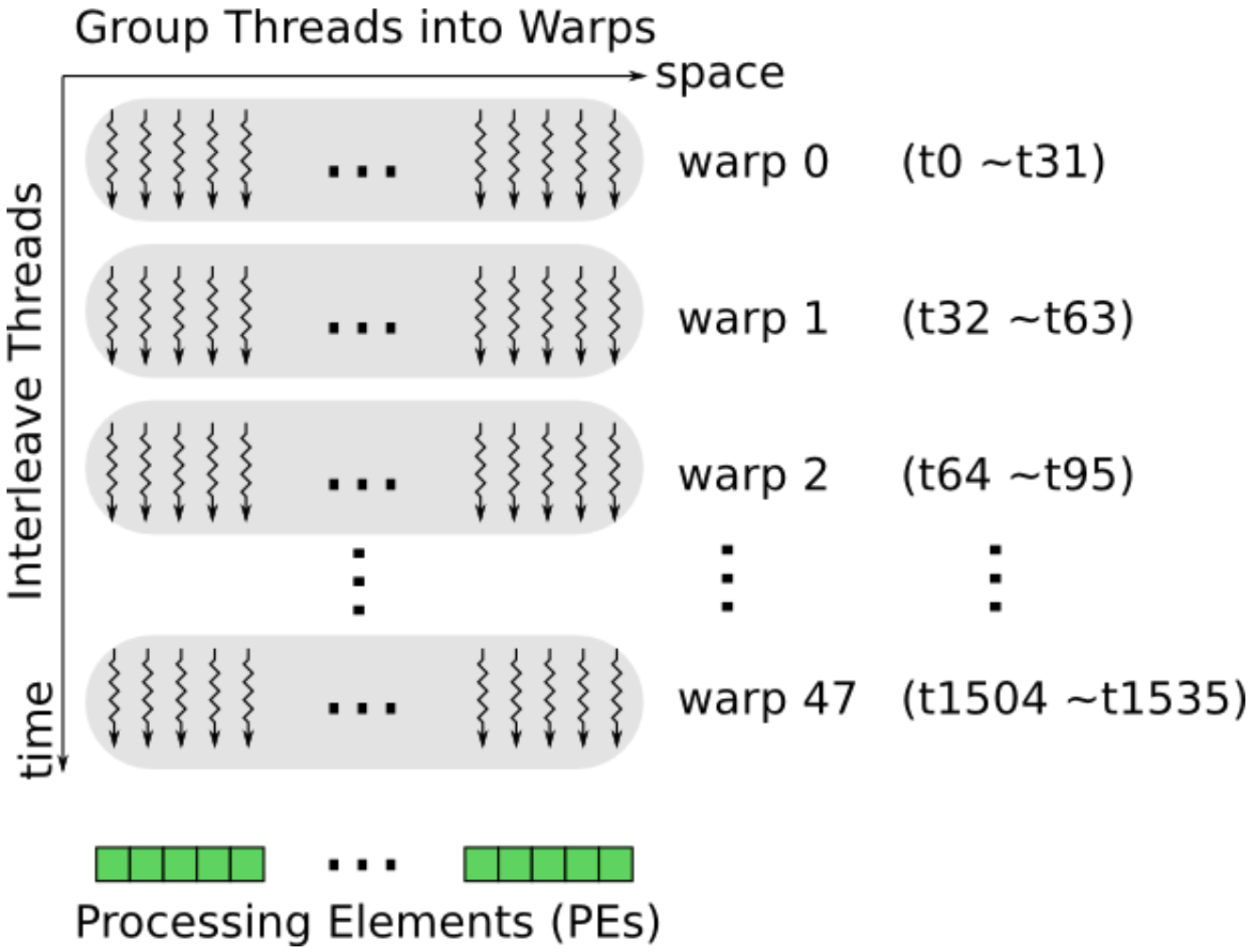
Single Instruction Multiple Thread (SIMT)

Hide vector width using scalar threads.

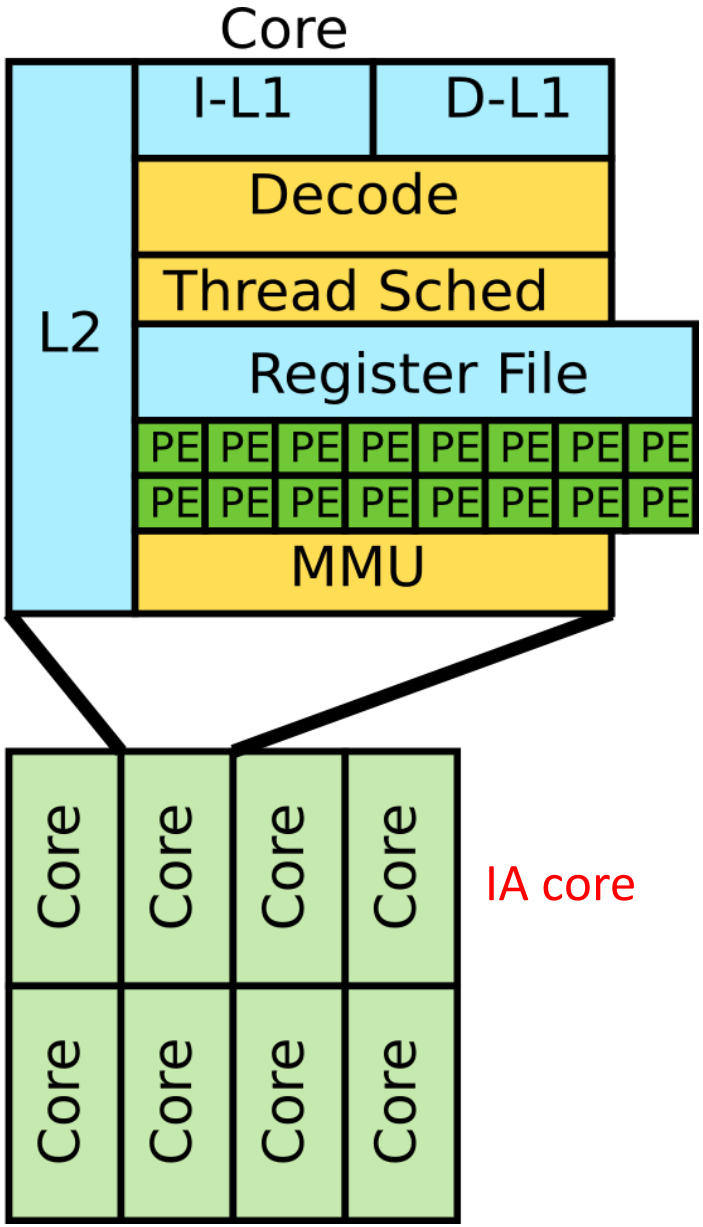


Example of SIMT Execution

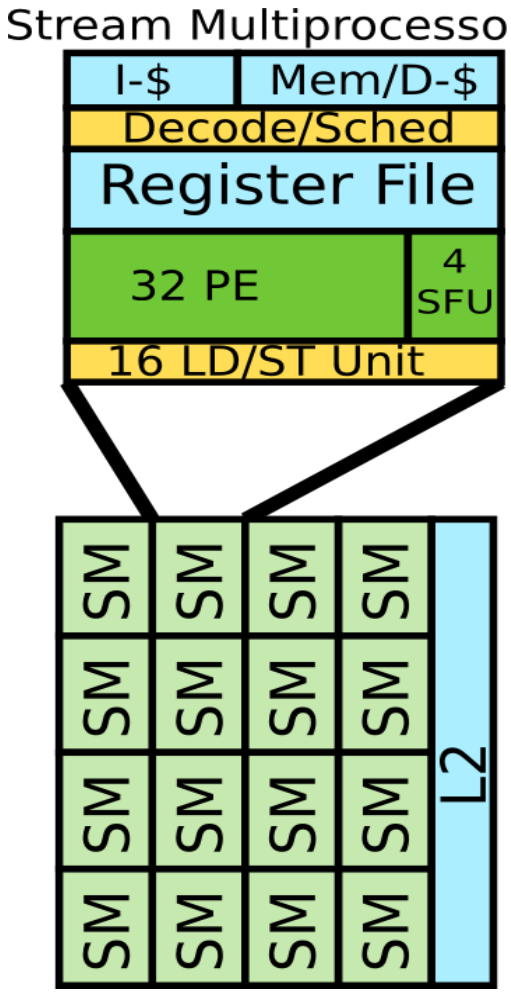
Assume 32 threads are grouped into one warp.



Step 3: Simple Core



The Stream Multiprocessor (SM) is a light weight core compared to IA core.



Light weight PE:
Fused Multiply Add (FMA)

SFU:
Special Function Unit

NVIDIA's Motivation of Simple Core

"This [multiple IA-core] approach is analogous to trying to build an airplane by putting wings on a train."

--Bill Dally, NVIDIA



Throughput Oriented Architectures

1. Fine-grained interleaved threading (~2x comp density)
2. SIMD/SIMT (>10x comp density)
3. Simple core (~2x comp density)

Key architectural features of throughput oriented processor.

ref: Michael Garland. David B. Kirk, "Understanding throughput-oriented architectures", CACM 2010. (link)

SIMD Architectures vs GPUs

- GPUs have more SIMD lanes
- GPUs have hardware support for more threads
- Both have 2:1 ratio between double- and single-precision performance
- Both have 64-bit addresses, but GPUs have smaller memory